



Research Paper

Reinforcement learning based robot navigation using illegal actions for autonomous docking of surface vehicles in unknown environments

Maria Inês Pereira^{*}, Andry Maykol Pinto

FEUP - Faculty of Engineering of University of Porto, Rua Dr. Roberto Frias, 4200 - 465, Porto, Portugal

INESC TEC - Institute for Systems and Computer Engineering, Technology and Science, Rua Dr. Roberto Frias, 4200 - 465, Porto, Portugal

ARTICLE INFO

Keywords:

Reinforcement learning
Actor-critic
Autonomous docking
Autonomous surface vehicle

ABSTRACT

Autonomous Surface Vehicles (ASVs) are bound to play a fundamental role in the maintenance of offshore wind farms. Robust navigation for inspection vehicles should take into account the operation of docking within a harbouring structure, which is a critical and still unexplored maneuver. This work proposes an end-to-end docking approach for ASVs, based on Reinforcement Learning (RL), which teaches an agent to tackle collision-free navigation towards a target pose that allows the berthing of the vessel. The developed research presents a methodology that introduces the concept of *illegal actions* to facilitate the vessel's exploration during the learning process. This method improves the adopted Actor-Critic (AC) framework by accelerating the agent's optimization by approximately 38.02%. A set of comprehensive experiments demonstrate the accuracy and robustness of the presented method in scenarios with simulated environmental constraints (Beaufort Scale and Douglas Sea Scale), and a diversity of docking structures. Validation with two different real ASVs in both controlled and real environments demonstrates the ability of this method to enable safe docking maneuvers without prior knowledge of the scenario.

1. Introduction

Offshore wind energy has become increasingly relevant for sustainable development due to the strong and steady wind gusts provided by maritime scenarios. Such harsh environments imply more frequent Operations and Maintenance (O&M) routines to ensure proper functioning of the wind turbines and adjunct structures, which are constantly subject to corrosion and fatigue. Inspection techniques can be enhanced through the utilization of autonomous vehicles, thus making the supervision process safer and more efficient. Autonomous Surface Vehicles (ASVs) are bound to be part of this paradigm shift towards automatized procedures in maritime environments (Pinto et al., 2021), but the technology behind them is currently lacking substantial development to enable a full-scale application of autonomous decision making (Leite et al., 2018). From perception systems with the required robustness for maritime scenarios (Pinto and Matos, 2020; Campos et al., 2021b; Claro et al., 2020), to navigation techniques that ensure obstacle avoidance (Campos et al., 2019; Silva et al., 2019), several challenges still need to be solved.

Docking is one of the primary functions of any vessel, allowing its recovery for charging and maintenance procedures, and one particularly lacking technological advancements in the automation subject (Bartels et al., 2022). This is intrinsically a critical maneuver, and

the complexity of seaports and harbours further adds to the propensity for accidents (Darbra and Casal, 2004). The scarcity of research in this topic still holds back the transition from the scientific domain to the technological one for autonomous docking. In these circumstances, Reinforcement Learning (RL) appears as a mechanism to accomplish autonomy in complex robotic problems (Garaffa et al., 2021). This Machine Learning approach aims to make an agent learn a task by interacting with an environment, without receiving explicit examples (Sutton and Barto, 2017). The agent experiments different actions and, through the maximization of the reward that it receives for each decision, it attempts to learn an optimal policy (π^*) that maps environment states (s_t) to actions (a_t). This learning paradigm resembles the way humans learn, i.e., by trial-and-error, and therefore it is more suitable for decision-making problems such as the one at hand. RL also differs from more traditional methods, such as fuzzy logic (Valdez et al., 2020), in the sense that it does not require explicit modelling of every possible state the agent might encounter (Capra et al., 2023). This is a fundamental characteristic for dealing with unknown scenarios, which is one of the objectives of this research. Additionally, RL is more suitable for dealing with continuous data scenarios (Zhang et al., 2023b), as is the case of autonomous vehicles navigation.

^{*} Corresponding author at: FEUP - Faculty of Engineering of University of Porto, Rua Dr. Roberto Frias, 4200 - 465, Porto, Portugal.

E-mail address: maria.i.pereira@inesctec.pt (M.I. Pereira).

This research provides an intelligent agent built on RL for surface vehicles to dock autonomously in harbours of different configurations and sizes, using only their onboard perception system to guide them into unknown environments. The focus of this work lies in bridging those perception abilities to autonomous navigation skills, that is, it aims to enable the agent to learn and to make decisions directly supported by the information collected from the vehicle's perception system. This intelligent agent handles decision-making processes for motion planning that simultaneously contribute towards achieving a desired docking pose and ensuring obstacle avoidance. Several challenges that impose significant limitations on the manoeuvrability and perception capabilities of ASVs are addressed in this work, such as performing the docking maneuver in unknown scenarios, in close range to harbour structures, and constrained by phenomena like wind and ocean waves. These constraints can be successfully tackled with an RL-based method, given that it allows the agent to improve its capability of adaptation to unseen and unmodelled conditions. The contributions of this article include:

- An RL-based framework for autonomous docking of a surface vehicle, which leverages the agent's perception abilities for high-level motion planning that enables reaching a target pose within a docking structure while avoiding collision. Unlike other methods which are given intermediate poses obtained from pre-made maps of the scenario, the RL agent developed in this work operates without previous knowledge of the environment and relies only on the final target pose and on the vehicle's onboard perception system;
- A new concept entitled *illegal actions* used during the agent's training process to increase the coherence of actions to be selected at each iteration step by disabling opposite actions over a time period, and thus optimize the exploration. This methodology also helps to consolidate the potential of Actor-Critic frameworks;
- Extensive experiments using both simulated and real scenarios, as well as two different ASVs, that validate the system's docking capabilities with a success rate of 96.4% and evaluate its robustness to different degrees of environmental constraints and various docking structure configurations, thus proving the method's resilience to unknown conditions. Moreover, the concept of *illegal actions* accelerates the training process by 38.02%.

The remainder of this article is structured as follows. Section 2 portrays the current state of the art on RL-based robotic applications, in particular regarding maritime scenarios and docking for ASVs. Section 3 begins with a brief description of fundamental RL concepts for understanding this work, followed by the characterization of the environment representation, action space and reward function, along with other methodologies that add to this paper's contributions. The results of the proposed method are then presented and discussed in Section 4. Finally, Section 5 summarizes the main conclusions of this work.

Notations: Lower case boldface letters represent vectors, whereas upper case boldface letters denote matrices. $E[a]$ denotes the expectation of a , and $\hat{b}$ represents the estimate value of variable b .

2. Related work

The earliest methods that combine RL with robotics are related to general navigation tasks, such as obstacle avoidance or adhering to a predefined trajectory (Cebollada et al., 2021). These first efforts often considered simplified state representations or assumed known positions and velocities for surrounding objects (Duguleana and Mogan, 2016). In the meantime, RL-based research in robotics has undergone considerable progress, contributing to the development of new and more sophisticated methods (de Bruin et al., 2018; Pfeiffer et al., 2018). Nonetheless, these advances have not been sufficient for safe deployment of navigation techniques fully based on RL, as many approaches

still rely on combining this Machine Learning variant with traditional path planning methods (Chang et al., 2021).

This lack of adequate research is especially valid for slow-changing industries, such as the maritime sector, which has only recently seen an expansion of studies built on RL. One of the first proposed methods in this subject was the work of Anderlini et al. (2019), who developed an RL algorithm to control the docking process of an Autonomous Underwater Vehicle (AUV) onto a charging station. The performance of this method was compared with that of established optimal control strategies in a simulated environment, having outperformed parametric optimal control but underperformed when compared to a PID-based approach. Also in the underwater domain, Cao et al. (2019) ventured into unknown scenarios with the task of target search. Although more daring, this work made use of a combination of RL with traditional exploration methods and relied on a simplified grid map to represent the search environment. A few other AUV-based approaches have been developed more recently, in particular towards path planning tasks, but performance assessment was limited to simulated scenarios (Chu et al., 2023; Hadi et al., 2024).

RL studies regarding surface vehicles suffer from similar limitations. The majority of these methods focus on ensuring trajectory tracking of an ASV, such as the work of Zhang et al. (2020), which means they are based on the existence of a predefined path. Zhang combined conventional model-based control with Deep RL, where the first component provided closed-loop stability, whereas the RL module counteracted the modelling uncertainties. Despite the innovation, this method was only tested in simulated environments without considering any environmental disturbances. By contrast, Woo and Kim (2020) validated their work for collision avoidance of an ASV in real-world scenarios. The authors adopted a quite high-level approach for the vehicle's behaviour such as "path following, starboard avoidance, and port avoidance instead of instant low-level action such as steering control command" (Woo and Kim, 2020). This method achieved comparable performance to traditional methods, but a number of limitations were acknowledged by the authors, such as long training duration (hundreds of hours) caused by a high-dimensional state representation. The main drawback of this work, however, was the accuracy loss observed in the real-world deployment of the algorithm. Simulation-only training with disregard for environmental constraints was the likely cause for the reduction of the method's reliability.

In 2020, Meyer et al. (2020b) proposed a method to handle both path following and collision avoidance using Deep RL. This work adopted a continuous action space and a set of 225 rangefinders to assess the collision risk surrounding the vehicle, considering only static obstacles. The framework included a trade-off parameter used for the computation of the total reward, that gave more or less importance to either the path following or the collision avoidance task, and which was fine tuned by the authors. Despite having achieved a success rate close to 100%, this approach was only tested in simulated scenarios and did not consider wind, waves or other external disturbances to the vessel. Also in 2020, Meyer et al. (2020a) improved on this first approach by considering moving obstacles and complying with the International Regulations for Preventing Collisions at Sea (COLREGs). In spite of not considering maritime constraints, this work was validated in scenarios simulated more realistically, using a digital reconstruction of the Trondheim Fjord in Norway. The work of Lin et al. (2023) considers the effect of current flows, despite resorting to simulated constraints.

Other methods demonstrate further limitations in deploying an RL agent for maritime tasks, such as considering only tests in open sea (Chun et al., 2021), or relying on conventional control methods (Zhang et al., 2022). Wang et al. (2021a,b) produce more sophisticated approaches towards tracking control that are capable of dealing with vehicles of unknown dynamics and in realistically simulated environments. Lambert et al. (2021) provide an interesting insight on dealing with the challenge of gathering data from maritime scenarios. To achieve collision avoidance for an ASV, they first trained an RL

algorithm on a simulated ground environment and then conducted a domain transfer process to marine scenarios with minimal further training. This strategy was made possible by adopting high-level actions and by “leveraging existing path planning and proven low-level control methodologies that are utilized in multiple domains” (Lambert et al., 2021). Real-world tests in both ground and maritime environments proved the potential of the proposed methodology but also demonstrated the need for further research so that the method can be generalized to more complex settings, with dynamic obstacles, for example. The methodologies described thus far require at least partial knowledge of the environment, given that most rely on a path constructed from way-points known *a priori*. Amendola et al. (2020) took a step further from such assumptions by considering a path divided into segments and taking into account environmental conditions for proposing an Asynchronous Deep RL method for controlling a ship’s movement. In this approach, each path segment corresponded to an episode, which meant that, in fact, this strategy approximates this problem to the trajectory tracking task. Some of the described methodologies focus on using RL to replace optimal control processes (Zhang et al., 2022; Wang et al., 2023a), namely non-linear control (Rigatos et al., 2017; Roman et al., 2022), whereas others, such as the work proposed, employ it to achieve high-level decision making (Wang et al., 2023c; Xia et al., 2023; Zhao et al., 2022).

A successful docking maneuver of an ASV needs to handle all of the limitations described above, along with further challenges such as navigation in close range to infra-structures and other vehicles, and the requirement of holding the desired pose within the docking structure. The level of difficulty of this task accounts for the lack of research in making this an unmanned procedure. Nonetheless, a few recent works demonstrate that a shift towards autonomous docking is currently in motion, with advanced perception systems being developed to assist in this task (Kim et al., 2020; Pereira et al., 2021b; Volden et al., 2022, 2023). Some conventional approaches towards automatic docking have been proposed, but they either disregard real-world constraints such as wind and waves (Li et al., 2020), or are built on optimization-based trajectory planners and thus rely on previously mapped environments (Martinsen et al., 2020; Wang et al., 2023b; Cai et al., 2023). Nearly all of these approaches lack successful real-world deployment (Liu et al., 2024; Zhang et al., 2023a; Leite et al., 2019).

RL-based approaches in this topic are very scarce, which demonstrates that Artificial Intelligence (AI) is still in a very early stage concerning critical tasks in the maritime environment. Existing works consider simplified formulations of the problem in 2D simulated scenarios (Holen et al., 2022; Bjering Strand, 2020). Recently, Higo et al. (2023) applied a trajectory-tracking methodology based on RL to achieve automatic berthing from an initial waypoint-based path computed on a previously mapped harbour. Another work that stands out is that of Rørvik (2020), who developed an automatic docking procedure for ASVs built on RL that achieved good performance and produced efficient trajectories. The author also considered a simplistic conceptualization of the problem, which consisted of the vessel approaching the shore and holding a desired pose, without any other obstacles besides the shore. Moreover, the algorithm was trained and validated in a simulated environment, considering ocean currents of fixed amplitude and direction, but no wind nor waves. Nonetheless, Rørvik’s work is a great contribution for the integration of AI in the maritime sector and it inspired other authors to study the potential of RL for solving this kind of problems. Both Løver et al. (2021) and Gjørsum et al. (2021) picked up Rørvik’s RL algorithm for automatic docking and developed methods to interpret the behaviour of the trained agent, through explainable AI, for example, which demonstrated that RL does not have to be a “black-box” approach as it is often claimed to be and that it can indeed be used for critical operations such as docking maneuvers.

3. Deep reinforcement learning for autonomous docking

This section presents a brief description of some fundamental RL concepts that provide the groundwork for this research. The developed methodology is explained in detail, from the state representation and the action space to the reward function. The section closes with a discussion on how to balance exploration and exploitation, which results in the introduction and characterization of the *illegal actions* concept.

3.1. RL fundamentals

RL frameworks are usually categorized into policy-based or value-based (Hameed et al., 2021). Models in the first category try to directly optimize the policy, whereas models in the second one attempt to evaluate the expected future return, i.e., learn a value function, and extrapolate the policy from it. Actor-Critic methods combine both of these approaches (Grondman et al., 2012) and have shown promising results (Alexandru Zamfirache et al., 2023). In this type of framework, the Actor represents the policy being optimized, which means it receives the current state of the environment and outputs an action to be taken. On the other hand, the Critic represents the value function that is being learnt. Simply put, it provides feedback on the actions taken by the Actor by evaluating the goodness of the current state. The Actor then updates its policy according to the critic’s feedback. Both the Critic and Actor functions are parameterized with neural networks.

This work makes use of a variant of Actor-Critic methods called Advantage Actor-Critic (A2C) (Mnih et al., 2016). Advantage represents the appropriateness of a certain action in a certain state. In other words, it shows how much better it is to take a specific action compared to the average, in a given state. Eq. (1) shows that the Advantage value (A) is derived from the state-action value function (Q) using the state value function (V) as baseline:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t). \quad (1)$$

In order not to rely on a third neural network to estimate the Q -value function, and thus compromise efficiency, the previous equation can be rewritten by using the relationship between the Q and the V values from the Bellman optimality Eq. (2). Eq. (3) is thus obtained, depending only on the Value function (V) given by the Critic, on the rewards (r) and on the discount factor (γ), that varies between 0 and 1:

$$Q(s_t, a_t) = \mathbb{E}[r_{t+1} + \gamma \cdot V(s_{t+1})], \quad (2)$$

$$A(s_t, a_t) = r_{t+1} + \gamma \cdot V(s_{t+1}) - V(s_t). \quad (3)$$

The Advantage value is leveraged to update the Actor’s network, and there are a number of strategies to do such. Proximal Policy Optimization (PPO) (Schulman et al., 2017) is one of the most efficient. It represents an improvement on Trust Region Policy Optimization (TRPO) (Schulman et al., 2015), which was first developed as a method to prevent destructively large policy updates. The general optimization objective of policy gradient methods can be considered as depicted in Eq. (4):

$$L^{PG}(\theta) = \mathbb{E}_t[\log(\pi_\theta(a_t|s_t))\hat{A}(s_t, a_t)], \quad (4)$$

where L^{PG} is the policy gradient loss, $\log(\pi_\theta(a_t|s_t))$ are the logarithmic probabilities from the output of the policy network, and $\hat{A}(s_t, a_t)$ is the estimated Advantage value. These are highly dependent on the knowledge gathered by the agent in the environment. Due to the stochastic component of the agent’s exploration, there can be a large deviation between historical experience and the current policy, which leads to model instability. To restrain this effect, TRPO introduces the probability ratio between the current policy and the previous one, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$:

$$L^{CPI}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}(s_t, a_t) \right], \quad (5)$$

as well as a hard constraint to prevent the gradient step from moving the current policy too far away from the original policy, thus defining a “trust region” represented by a threshold δ , known as the trust radius:

$$\hat{\mathbb{E}}_t[KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t)]] \leq \delta, \quad (6)$$

where KL represents the Kullback–Leibler divergence, which can be solved approximately using conjugate gradient. In an attempt to reap the benefits of TRPO’s approach while keeping a first-order optimization problem, PPO discarded the constraint and presented a clipped objective that penalizes any update that moves the probability ratio too far from 1, by clipping it within an interval $[1 - \epsilon, 1 + \epsilon]$, where ϵ is the clipping factor. The gradient loss is then obtained by computing the expectation over the minimum between the clipped and the unclipped objective as depicted in Eq. (7):

$$L^{clip}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 + \epsilon, 1 - \epsilon)\hat{A}_t)]. \quad (7)$$

PPO includes an additional entropy term (c_1) within the expectation computation to ensure sufficient exploration by the agent.

Given the efficiency of the described algorithms, this work adopted an A2C architecture using PPO to update the Actor’s network. The Critic’s value function is optimized simply by using the discounted sum of rewards as target to compute a loss based on the mean squared error. The following sections contextualize this approach within the challenge of autonomous docking, in particular by describing the design of the reward function and by proposing changes to the aforementioned optimization process to speed up convergence in problems with large and disparate action spaces.

3.2. State and action spaces for ASV docking

The scope of autonomous docking in this research extends to commercial harbours, which are highly complex environments that may not be known *a priori*. To achieve a thorough representation of this kind of scenario for a successful RL framework, the state space combined navigation and perception-based features: $\mathcal{S} = [f_{nav}, f_{pre}]$. The former set of features represent the position, orientation and velocity of the ASV relative to its target pose (Fig. 1), which are given by sensors such as GPS/RTK and IMU, whereas the perception-based features comprise measurements obtained by visual or range sensors, such as a 3D LiDAR, to detect nearby objects and avoid collision. Both sets of observations are described in the present section.

This framework is targeted at differential vessels, with two propellers that allow surge and yaw movement, which represents the majority of maritime vehicles. As such, the target pose is characterized by three parameters: the x and y coordinates, and the yaw angle (ψ). Consequently, the first three elements of the state space are the Cartesian distance between the origin of the vessel and this target position in the x and y axes, and the difference between its heading and the desired one. Additionally, the roll (ϕ) and pitch (θ) angles were taken into consideration to detect safety-compromising events such as the boat turning over. Finally, the navigation-based features include the vessel’s linear and angular velocities (v_{lin} and v_{ang}), that correspond to the surge velocity and the yaw rate, respectively. These two state elements should be close to zero in the proximity of the target pose so that the docking episode can be successfully terminated. In addition, they allow for speed management when the boat is navigating in close range to structures. The navigation features vector is then given by:

$$f_{nav} = [x, y, \psi, \phi, \theta, v_{lin}, v_{ang}]. \quad (8)$$

The presented navigation features are sufficient for the ASV to reach a target pose within an obstacle-free scenario. However, the docking process is carried out within a harbour where there are multiple vessels and various structures such as docks, among other types of objects. Therefore, the RL agent should be able to navigate through this environment without colliding with these entities. Collision avoidance cannot

be achieved without an accurate 3D perception of a 360° horizontal Field-of-View (FoV) around the vessel, which can be obtained from sensors onboard the vehicle that produce point clouds representative of the scenario. The scene reconstruction provided by those point clouds can be leveraged to achieve high-level situational awareness (Leite and Pinto, 2024), which in turn is fundamental for avoiding obstacles. In this work, such awareness is modelled by assessing the collision risk around the vessel.

Firstly, a laser scan containing 360 distance measurements that fully cover the scenario surrounding the vessel is obtained from each point cloud. This step transforms a very unstructured data representation into a comprehensible image suitable for analysing the collision risk, as depicted in Fig. 2. Each ray represents the distance from the origin of the vessel to the surface of the nearest obstacle in the corresponding direction and at the specified height range.

This is a more lightweight representation than a point cloud, but it still corresponds to an overly large state space and, as such, its direct processing by the Actor’s and Critic’s networks would likely lead to convergence difficulties, given the “curse of dimensionality” (Sutton and Barto, 2017). To reduce complexity and, thus, not compromise efficiency, it is necessary to downsample this information by grouping the 360 measurements into a few sectors and then computing an average collision distance representative of each sector. Considering the vessel’s degrees of freedom, 8 main sectors were deemed sufficient for a thorough coverage of the collision risk, as illustrated in Fig. 2, each with a range of $\frac{\pi}{4}$ rad. The collision distance of each sector is given by the average of the 20% lowest distances of the corresponding sector. Then, a collision risk parameter that varies between 0 and 1 is computed according to Eq. (9):

$$risk_{collision_\beta} = 2^{-k_s(ObstDist_\beta - Tol)}, \beta \in \{0, \dots, 7\}, \quad (9)$$

where $ObstDist_\beta$ is the collision distance of sector β , Tol is a tolerance value for the collision detection to be triggered slightly before the actual collision happens, thus giving the risk analysis a more conservative dimension, and k_s is a scaling factor. The perception-based feature vector is then given by concatenating the 8 risk parameters:

$$f_{pre} = [risk_{collision_\beta}], \beta \in \{0, \dots, 7\}. \quad (10)$$

Fig. 3 illustrates two examples of the collision risk analysis. The left column of the figure presents a model of an ASV positioned in two different stages of the docking process. The right column depicts the point cloud obtained by the LiDAR onboard the vehicle and a set of green arrows that represent the collision risk level in the 8 main sectors. The length of each arrow varies directly with the collision risk and inversely with the distance to the nearest obstacle. An arrow of length zero corresponds to the inexistence of obstacles in that direction. This method for the analysis of the collision risk can be easily replicated using other types of sensors such as rangefinders, radars and depth cameras. In addition, it works for any number of obstacles, of any size and shape, while maintaining a constant state space dimension.

The ASV’s motion can be discretized into a set of actions representing translational and/or rotational movement. These actions were selected to address high-level behaviour such as “moving forward” or “rotating left”. By not setting propeller velocity commands as direct outputs, but linear and angular velocities, this work can be easily adapted to vehicles with different propeller configurations. Both forward and backward motion are allowed so as not to limit the vessel’s manoeuvrability. Actions combining translation and rotation are included to smooth the variation of the propellers’ velocities. The introduction of two velocity levels (fast and slow) for some actions enables the vessel to manage its speed throughout the different stages of the docking process. A complete list of the action space¹ and corresponding linear and angular velocities is presented in Table 1.

¹ <https://youtu.be/1BqNzelp6qI>

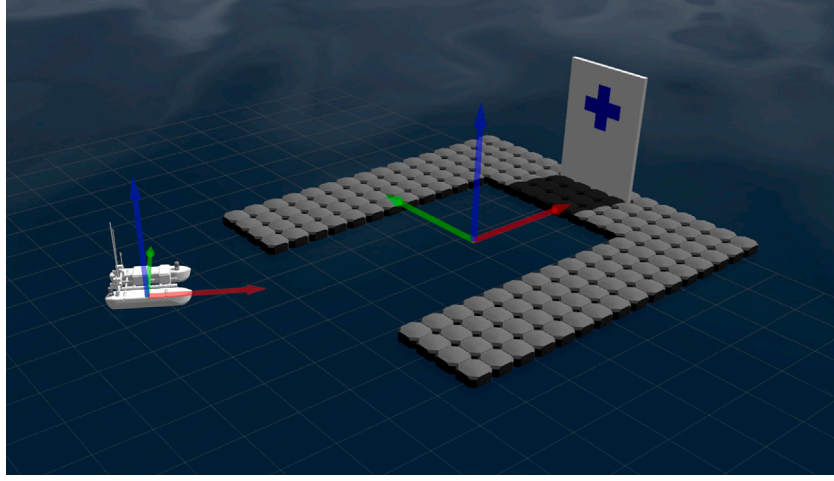


Fig. 1. Target pose within the docking structure that the ASV has to reach in order to terminate an episode with success.

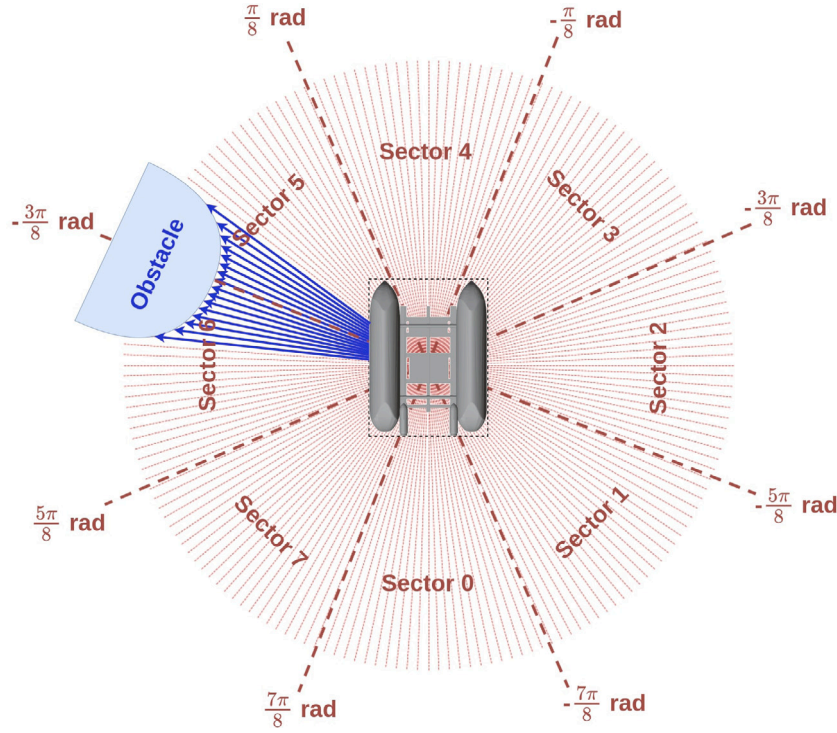


Fig. 2. The 360 distance measurements are grouped into 8 sectors centred on the angles from $-\pi$ to $\frac{3\pi}{4}$ rad with a step of $\frac{\pi}{4}$ rad.

3.3. Reward function

The maximization of the reward is the ultimate objective of the agent for it to successfully learn the required task. This research proposes a reward function based both on sparse and dense components. The attribution of a binary reward at the end of each episode, based on the successful completion of the task or not, would be insufficient and would likely prevent the agent from learning enough information to achieve its objective. An efficient solution is reward shaping (Hu et al., 2020), which consists of reshaping the original final reward into smaller continuous rewards that give feedback on intermediate actions. The random exploration employed in the PPO's training procedure further increases the need for continuous assessment. As such, the proposed reward is divided into two elements as presented in Eq. (11):

$$r_{total}(t) = r_{sparse}(t) + r_{dense}(t). \quad (11)$$

The sparse component of the proposed reward function is built on specific events that lead to the termination of an episode. These events, which are listed below, may represent either a positive ($\oplus$), negative ($\ominus$) or neutral ($\odot$) outcome:

- $\oplus$ *target* - the vessel reaches the target pose;
- $\ominus$ *collision* - the vessel collides with one or more obstacles;
- $\ominus$ *turn_over* - the vessel turns over;
- $\ominus$ *boundary* - the vessel overcomes a distance boundary defined to prevent exploration too far from the docking structure;
- $\odot$ *time_over* - the episode reaches its maximum allowed duration.

Both the positive and negative events are considered for computing the sparse reward component, as demonstrated in Eq. (12), where $\lambda_i(t)$ are binary variables that take value 1 if the corresponding event happened and 0 otherwise, and r_i are the rewards associated to those

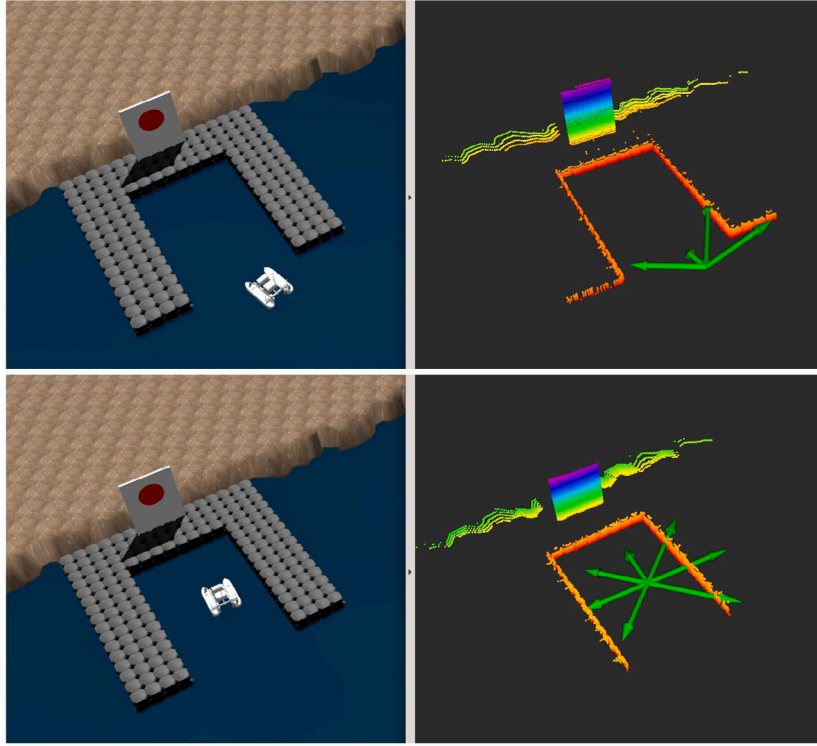


Fig. 3. Example of the collision risk analysis in two different stages of the docking process. The images on the left column depict the two stages in simulation. The images on the right column display the corresponding point clouds and a set of green arrows representing the collision risk in the 8 main directions. Arrow length varies directly with the risk. An angle without arrow corresponds to zero risk.

Table 1

Action space: linear and angular velocities per action.

#	Action	Linear velocity (m s ⁻¹)	Angular velocity (rad s ⁻¹)
0	∅ Do nothing	0	0
1	↑ Move forward slowly	$k \cdot v_{lin}$	0
2	↑ Move forward fast	v_{lin}	0
3	↶ Move forward while turning left	$\frac{v_{lin}}{2}$	$\frac{v_{ang}}{2}$
4	↷ Move forward while turning right	$\frac{v_{lin}}{2}$	$-\frac{v_{ang}}{2}$
5	↶ Rotate left	0	v_{ang}
6	↷ Rotate right	0	$-v_{ang}$
7	↓ Move backward slowly	$-k \cdot v_{lin}$	0
8	↓ Move backward fast	$-v_{lin}$	0
9	↶ Move backward while turning left	$-\frac{v_{lin}}{2}$	$-\frac{v_{ang}}{2}$
10	↷ Move backward while turning right	$-\frac{v_{lin}}{2}$	$\frac{v_{ang}}{2}$

v_{lin} and v_{ang} are the maximum allowed linear and angular velocities and k is a regulating velocity factor ($k = 0.3$ in this case).

events. The neutral event, *time_over*, does not affect the reward received by the agent, but it simply limits the duration of each episode.

$$r_{sparse}(t) = \sum_i \lambda_i(t) r_i, \quad i \in \{target, collision, turn_over, boundary\}, \quad (12)$$

In this work, $r_{target} = 5$, $r_{collision} = r_{turn_over} = -10$, and $r_{boundary} = -5$. The criteria that determine whether $\lambda_i(t)$ is 1 or 0 for each event are depicted in Eq. (13) to (16).

Parameter λ_{target} takes value 1 if the agent is considered to have achieved its target, i.e., if its distance to the docking position, its heading difference and its linear and angular velocities are all under specific thresholds represented by ϵ , as depicted in Eq. (13). The variable *dist* is given by the Euclidean distance to the target: $dist(x, y)$ where x and y are the first two elements of the state space, i.e., the distance between the current and target positions in the x and y coordinates, respectively.

$$\lambda_{target} = (dist < \epsilon_{dist}) \wedge (\psi < \epsilon_\psi) \wedge (v_{lin} < \epsilon_{v_{lin}}) \wedge (v_{ang} < \epsilon_{v_{ang}}). \quad (13)$$

Parameter $\lambda_{collision}$ is set to 1 if the collision event is detected, which happens whenever the collision risk is greater than or equal to 1 for any of the considered directions (Eq. (14)):

$$\lambda_{collision} = \neg(risk_{collision_\beta} < 1), \forall \beta \in \{0, \dots, 7\}. \quad (14)$$

Parameter λ_{turn_over} represents the event of the vessel turning over which is detected by the pitch or roll angles taking values greater than a threshold (Eq. (15)):

$$\lambda_{turn_over} = (\phi > \epsilon_\phi) \vee (\theta > \epsilon_\theta). \quad (15)$$

Finally, $\lambda_{boundary}$ is equal to 1 if the vehicle's distance to the target is greater than the boundary's radius, B , (Eq. (16)):

$$\lambda_{bound} = (dist > B). \quad (16)$$

The dense reward component is based on state parameters that are continuously assessed during the episode. As shown in Eq. (17), there are four elements to these continuous rewards: based on the distance between the vessel and the target (*dist*), its orientation (*orien*), velocity (*vel*) and collision risk (*risk*), each one varying within $[-1; 0]$. These reward components are described throughout this section and illustrated in Fig. 4.

$$r_{dense}(t) = r_{dist}(t) + r_{orien}(t) + r_{vel}(t) + r_{risk}(t). \quad (17)$$

The first reward element is related to *dist*. Naturally, the agent's objective is to minimize this variable and therefore $r_{dist}(t)$ should increase as the distance decreases. Fig. 4(a) illustrates this relationship. This reward's curve assumes an exponential form when *dist* is lower than 10m and a linear one otherwise, as depicted in the figure. This differentiation is important to heighten the sensitivity to distance variations when the ASV is near the target.

The second element is based on the yaw of the vessel relative to the docking target referential. This reward component ($r_{orien}(t)$) is divided into two stages that are dependent on *dist*, and which are proposed

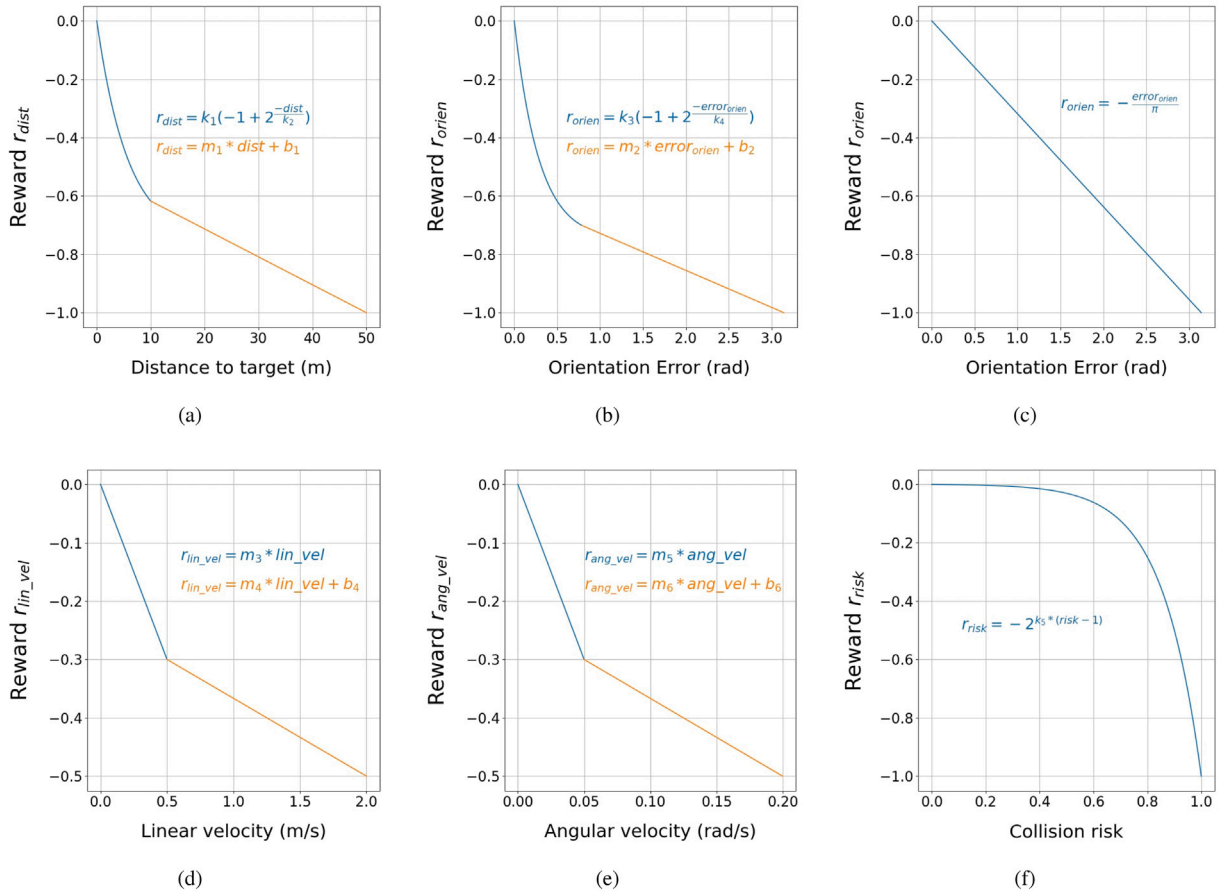


Fig. 4. Dense reward components according to: distance to target (a), orientation error when the distance is less (b) and more (c) than 5 m, linear (d) and angular (e) velocity, and collision risk (f). In this case, $k_1 = 4$, $k_2 = k_4 = 0.75$, $k_3 = 0.2$, $k_5 = 10$, $m_1 = -0.0096$, $m_2 = -0.127$, $m_3 = -1.2$, $m_4 = -0.267$, $m_5 = -12$, $m_6 = -2.67$, $b_1 = -0.5218$, $b_2 = -0.601$, $b_4 = -0.467$.

to help the agent direct itself towards the docking structure initially and only seek the desired heading in the final stage. If the agent is less than 5 m away from the target, the orientation reward measures how close the vessel's yaw is to the target one ($error_{orien}$). This error is normalized in the interval $[-\pi; \pi]$. Fig. 4(b) presents the reward for this case, considering a maximum rotation error of π rad, and an evolution for $r_{orien}(t)$ similar to that of $r_{dist}(t)$, i.e., by combining an exponential component for lower orientation errors and a linear one for higher errors. If the agent is more than 5 m away from the target position, then the orientation error ($error_{orien}$) is computed relative to an angle that ensures the front of the vessel is directed to that target point, also going through normalization in the interval $[-\pi; \pi]$. This way, the agent learns to prioritize the minimization of the distance at this stage. As depicted in Fig. 4(c), this reward varies linearly with $error_{orien}$.

In the proposed reward function, the velocity-based component ($r_{vel}(t)$) is only considered during the final docking stage, which begins when the agent is less than 5 m away from the docking position. This constraint was introduced because the purpose of this reward is to help the agent learn to slow down when close to the docking structure and to fully stop when it reaches the target. Figs. 4(d) and 4(e) illustrate the evolution of the linear ($r_{lin_vel}(t)$) and angular ($r_{ang_vel}(t)$) velocity rewards, respectively. These are composed of two stages according to the velocity values: the first stage is steeper than the second one, similarly to previously mentioned rewards. However, in this case, the first stage is also a linear function instead of an exponential one to soften the deceleration. Both $r_{lin_vel}(t)$ and $r_{ang_vel}(t)$ vary from -0.5 to 0 and their sum corresponds to $r_{vel}(t)$, that varies between -1 and 0.

The final reward component is related to the collision risk. As previously described, the state space includes 8 measures of the collision risk, each corresponding to a specific direction. The exponential

function expressed in Fig. 4(f), that relates the collision risk level to its reward, is applied to each of the 8 measures. However, only the minimum value from these 8 rewards is included in the computation of the dense rewards, shown above in Eq. (17). This function's parameters were tuned according to several experiments that aimed to ensure the reward varied sharply enough so that the agent had time to react when drawing near an obstacle but also so that this reward component did not overcompensate the others when approaching the docking structure. Unlike other works (Meyer et al., 2020b,a), this approach does not give increased importance to the collision risk in front of the vessel because (i) backward movement is considered as well, and (ii) currents and waves are taken into account, which means collisions can happen sideways, especially when entering narrow structures such as docks.

Considering the numerical examples presented throughout this section, $r_{total}(t)$ may vary between -29 and 5. Nonetheless, these values are merely illustrative and can be easily adjusted for vehicles with different kinematic properties. To prevent unbalanced distributions of network weights (Dann and Thangarajah, 2021), the total reward value is down-scaled by a factor of 10.

3.4. Exploration vs. exploitation

At any given time-step during the training stage, the selection of the next action to take is based on a trade-off between exploration and exploitation, in an attempt to maximize the rewards. Exploitation is considered a greedy approach because it exploits the gathered knowledge to revisit positive rewards previously obtained. Exploration, on the other hand, attempts to expand that partial knowledge by

searching unknown state–action pairs and, thus, potentially receive larger rewards. PPO ensures this trade-off by sampling actions from the latest version of the agent’s network, which is used as a stochastic policy. This is achieved by using the network’s output probabilities to set a categorical distribution and then sampling an action from such distribution. This strategy causes the policy to be very random at first, thus prioritizing exploration, and to become progressively less random throughout the training phase as its knowledge gets broader and it leans to exploiting rewards already received.

Given the diversity of the action space considered in this work, exploration early in the training process may be hindered. A very random action selection process, as that of the first stages of training, implies that contrary actions, such as forward and backward movement or as left and right rotation, will have very similar probabilities of being sampled at any given state. This behaviour is undesired because it cancels the effect of actions that are followed by their contrary ones and it likely prevents the vessel from leaving its place, which blocks proper exploration and, thus, hinders the agent’s learning process. To facilitate exploration, this work establishes the concept of *illegal action* (IA), which determines that the agent should not take the opposite action of which it has just taken. As such, at each time-step, the proposed method disables the selection of a set of actions that are considered to be opposite of the most recently taken action. To achieve this deactivation, the current policy is modified as presented below:

$$\pi_{\theta}(a_t|s_t)' = \pi_{\theta}(a_t|s_t) \cdot (1 - \epsilon_{illegal}), \quad (18)$$

where $\epsilon_{illegal}$ is a parameter that takes value 0 for allowed actions and a value close to 1 for actions which are *illegal* at the moment. It is not exactly equal to 1 to avoid division by zero. This operation greatly reduces the probability of an *illegal action* being selected. This update is conducted for both the current ($\pi_{\theta}(a_t|s_t)$) and previous ($\pi_{\theta_{old}}(a_t|s_t)$) policies when determining the probability ratio $r_t(\theta)$. The set of $\epsilon_{illegal}$ parameters for all the actions composes a mask $\mathbf{M}$, which is employed when producing output probabilities for the actions through $\mathbf{M} \cdot \pi_{\theta}(a_t|s_t)$.

This deactivation of actions is enforced during a temporal window that is larger for action-pairs with higher opposition. For example, the action “moving forward” is completely contrary to “moving backward”, whereas “moving forward while turning left” only partially counters that movement. The level of action opposition can be obtained by $Op(a_{t-1}, a_t)$, where a_{t-1} corresponds to the action that has most recently been taken and a_t represents an action that can potentially be selected in the current time-step. The mapping between opposition levels and each action pair, $Op(a_{t-1}, a_t)$, is given by the look-up table depicted in Table 2, where two different levels are defined: b_1 and b_2 . Level b_2 corresponds to a higher opposition degree, whereas b_1 is assigned to action pairs of only partial opposition. Rotation-only actions are included in the latter group because they are essential for pose adjustment in this type of maneuver. The temporal window during which $\epsilon_{illegal}$ is not null, i.e., an action is considered *illegal*, is determined as below:

$$t = \begin{cases} [T, T + k_{b_1}] & \text{if } Op(a_{t-1}, a_t) = b_1, \\ [T, T + k_{b_2}] & \text{if } Op(a_{t-1}, a_t) = b_2, \end{cases} \quad (19)$$

where T is the current time, and k_{b_i} is the number of time-steps that correspond to the blocking temporal window of each level b_i . Given that b_2 is a level of higher opposition, it was specified that $k_{b_1} = \frac{k_{b_2}}{2}$.

This strategy of setting *illegal actions* reduces the randomness of the exploration by avoiding the selection of actions that do not contribute to the progression of the trajectory. During the training stage, committing the agent to its previous choices potentially helps the learning process because it teaches the agent about intermediate actions that lead to termination events and, thus, how to prevent them at an earlier stage. Once having learnt how to navigate and reach the target, restricting the agent’s actions is no longer necessary. As such, this method is not applied during the testing stage, giving the agent total freedom of movement.

Table 2

Opposition level of each action pair.

#	$\emptyset$	$\uparrow$	$\uparrow\uparrow$	$\curvearrowright$	$\curvearrowleft$	$\odot$	$\ominus$	$\downarrow$	$\downarrow\downarrow$	$\curvearrowright$	$\curvearrowleft$
$\emptyset$	–	–	–	–	–	–	–	–	–	–	–
$\uparrow$	–	–	–	–	–	–	–	b_2	b_2	b_1	b_1
$\uparrow\uparrow$	–	–	–	–	–	–	–	b_2	b_2	b_1	b_1
$\curvearrowright$	–	–	–	–	–	–	–	b_1	b_1	b_2	b_1
$\curvearrowleft$	–	–	–	–	–	–	–	b_1	b_1	b_1	b_2
$\odot$	–	–	–	–	–	–	b_1	–	–	–	–
$\ominus$	–	–	–	–	–	b_1	–	–	–	–	–
$\downarrow$	–	b_2	b_2	b_1	b_1	–	–	–	–	–	–
$\downarrow\downarrow$	–	b_2	b_2	b_1	b_1	–	–	–	–	–	–
$\curvearrowright$	–	b_1	b_1	b_2	b_1	–	–	–	–	–	–
$\curvearrowleft$	–	b_1	b_1	b_1	b_2	–	–	–	–	–	–

To discourage exploration far from the docking structure, the boundary mentioned in Section 3.3 was made dynamic. To recall its purpose, when the vessel overcomes this imaginary border the episode is terminated. By shrinking this boundary progressively throughout the episode, the training time will decrease because the algorithm will not accumulate large amounts of data from interactions within distant areas, which do not add as much to the agent’s learning process. This dynamic behaviour of the boundary is illustrated in Fig. 5, where B_t represents the boundary at time-step t and P_t is the agent’s position at that same instant.

4. Results and discussion

This section aims to evaluate the performance of the developed docking framework considering three testing scenarios: (i) a simulated environment in *Gazebo* using an emulated model of an ASV; (ii) a realistic controlled scenario with a prototype of a surface vehicle; (iii) a real harbour using a real ASV of higher Technology Readiness Level (TRL).

Scenario (i) features a simulated model of the SENSE ASV (Campos et al., 2021a), developed by INESC TEC. This is a differential surface vehicle of dimensions $1.5 \times 1.1 \times 1.5$ m (Length $\times$ Width $\times$ Height) and maximum velocity of 1.5 m s^{-1} , equipped with a 3D LiDAR (Ouster OS1-128), a GPS/RTK module and an IMU. This first scenario aims to provide a detailed analysis of the improvement given by the *illegal actions* concept, as well as the robustness of the algorithm considering different combinations of environmental constraints, such as wind and waves, and also by generalizing the scenario to other types of docking structures found in commercial harbours. The experiments of scenario (ii) took place in the water tank of the Civil Engineering Department of the Faculty of Engineering of the University of Porto, Portugal, using the real SENSE ASV to evaluate the performance of the proposed method with hardware in the loop while addressing near real-world conditions. Scenario (iii) was set in the ATLANTIS Test Centre (Pinto et al., 2021) in Viana do Castelo, Portugal, featuring a more robust craft, named Nautilus (Campos et al., 2024), which is a monohull vessel of dimensions $3.0 \times 1.4 \times 1.5$ m (Length $\times$ Width $\times$ Height) and maximum velocity of 3 m s^{-1} , equipped with a similar perception system to that of the SENSE ASV. This last set of experiments aim to validate the presented methodology in a complex scenario under harsher environmental conditions such as the ones presented by real ports.

Section 4.1 depicts the implementation details of the concepts proposed in this paper, while Sections 4.2, 4.3, and 4.4 present the results of testing scenarios (i), (ii), and (iii), respectively.

4.1. Implementation details

Regarding the A2C framework, both the actor’s and critic’s networks are composed of three fully connected layers each with 128 neurons, using the *Leaky ReLU* activation function. The critic’s network has

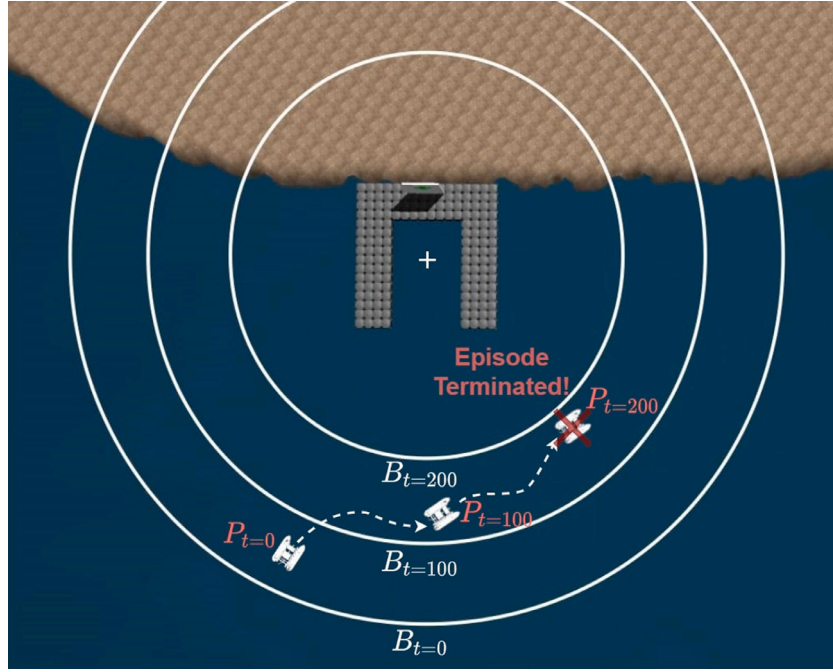


Fig. 5. Shrinking boundary that prevents the agent from moving too far from the docking structure. An imaginary border is drawn centred on the target position and its radius is progressively decreased. B_t represents the boundary at time-step t and P_t is the agent's position at that same instant. The episode is terminated whenever the agent's position is outside the boundary.

Table 3

Parameters used in the PPO's training algorithm.

Parameter	Value
Time-step duration (T)	200 ms
Maximum time-steps per episode (N_T)	400
Maximum linear velocity (v_{lin})	1.0 m s^{-1}
Maximum angular velocity (v_{ang})	0.4 rad s^{-1}
Replay buffer size (N_r)	10 episodes
Nr. epochs per update (N_e)	4
Actor learning rate (η_a)	5×10^{-4}
Critic learning rate (η_c)	1×10^{-3}
Discount factor (γ)	0.99
Clipping range (ϵ)	0.2
Entropy coefficient (c_1)	0.01

one output neuron, which corresponds to the state value. The policy network has a final layer with dimension equal to that of the action space, i.e., 11 in this case, which is then fed to a softmax layer to obtain probabilities for the actions.

Table 3 presents the parameter values used in training. An action given by the policy network is enforced on the vessel for the duration of a time-step (T), which in this implementation is 200 ms, given the low velocity of the SENSE ASV. An episode may last a maximum of 400 time-steps (N_T), that correspond to 80 s, because the docking process is a maneuver that is quite circumscribed in time. During the training stage, the value and policy networks are updated every 10 episodes (N_r). The rest of the table depicts the hyperparameters used in the PPO's training procedure. The critic's learning rate ($1e-3$) is greater than the actor's ($5e-4$) because the critic's feedback helps the agent's learning process, and therefore the optimization of the critic's network should be quicker. Table 4 presents the parameters of the episodes' termination criteria, presented in Eqs. (13) to (16). These parameters were kept for all the testing setups presented above, except for the maximum velocity values (v_{lin} and v_{ang}), which were reduced during the controlled-scenario experiments to 0.5 m s^{-1} and 0.2 rad s^{-1} , respectively, for safety purposes. As a consequence, the number of maximum time-steps per episode (N_T) was increased to 800.

Table 4

Parameters of episode termination criteria.

Parameter	Value
Distance threshold (ϵ_{dist})	1.0 m
Heading threshold (ϵ_θ)	0.2 rad
Linear velocity threshold ($\epsilon_{v_{lin}}$)	0.1 m s^{-1}
Angular velocity threshold ($\epsilon_{v_{ang}}$)	0.02 rad s^{-1}
Pitch threshold (ϵ_ψ)	0.5 rad
Roll threshold (ϵ_ϕ)	0.5 rad

Given the complexity of solving the docking task in unknown scenarios, the training framework was divided into two stages: approach and berthing (Nguyen, 2019). The first is the phase in which the vessel moves within the harbour up to a point close to the docking structure, whereas during the second it moves to the desired position within the structure to be later secured to the quay. Using a bottom-up strategy, the training algorithm begins with the second stage: berthing; which means the vessel begins each episode at a maximum distance of 5 m from the target position, and with a heading in the range $[-\frac{\pi}{4}; \frac{\pi}{4}]$ rad from the desired one. Throughout the training process, the range of the agent's initial pose is progressively expanded until it fully includes the approach stage (maximum distance of 16 m and heading range of $[-\frac{3\pi}{4}; \frac{3\pi}{4}]$ rad).

4.2. Simulation experiments

The purpose of the first testing scenario is to provide extensive validation of the proposed methodology in a low-cost and safe manner before real-world deployment. Initially, the potential of the *illegal actions* concept is demonstrated empirically, followed by experiments regarding the framework's robustness and generalization capabilities. Lastly, some analysis on safety and speed concerns is conducted against a state-of-the-art path planning method. These experiments were performed on a device with the following specifications: **CPU** - Intel® Core™ i5-10600K, **GPU** - NVIDIA GeForce RTX 2060 6 GB, **RAM** - 16 GB.

Table 5

Analysis of the *TF* and *IA* methodologies on the training time (number of interactions) over 10 training instances per methodology, with 2000 episodes per instance. One interaction corresponds to one instance of receiving the current state, computing an action to take based on that state and performing it.

Methodology	Training time (Interactions)		Improvement over <i>TF</i> (computed over the average training time)
	Average	Standard deviation	
<i>TF</i>	1.0192×10^5	1.0847×10^4	–
<i>IA</i> ($k_{b_2} = 2$)	6.7904×10^4	4.5189×10^3	33.38%
<i>IA</i> ($k_{b_2} = 4$)	6.3169×10^4	2.3312×10^3	38.02%
<i>IA</i> ($k_{b_2} = 8$)	6.8847×10^4	5.4646×10^3	32.45%

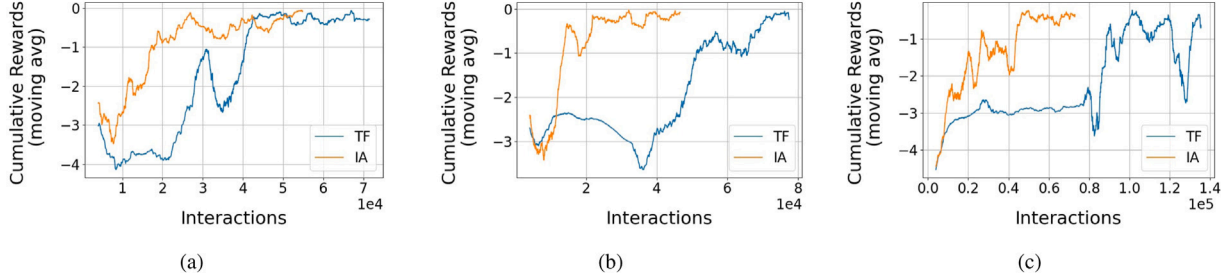


Fig. 6. Three examples of the comparison between the *IA* and the *TF* strategies. The charts present a simple moving average of the cumulative reward throughout the agent's training.

4.2.1. Illegal Actions (*IA*) vs. Total Freedom (*TF*)

The first analysis on this work's results concerns the *illegal actions* strategy, developed to improve the agent's exploration with action spaces as disparate as the one defined in this task. To assess this method's effective contribution to the training process, a comparison was made against a plain implementation in which the agent's actions are not limited, i.e., no modification was made to the PPO's algorithm. To enable a clear discussion of the results, the methodology using the *illegal actions* concept proposed in this work is hereafter referred to as *IA*, whereas the unmodified variant, that grants the vehicle with total movement freedom, is specified as *TF*. Within the *IA* variant, three different cases were studied regarding the duration of the blocking temporal window: $k_{b_2} = \{2, 4, 8\}$ time-steps. Greater blockage times were not explored because they would imply severe manoeuvrability limitations, taking into account a velocity of 1 m s^{-1} . Considering only the berthing stage for this analysis, ten training instances were conducted for each of the four strategies. Table 5 presents the average and standard deviation of the training time for each methodology. Time is quantified in interactions between the agent and the environment, in which one interaction corresponds to one instance of receiving the current state, computing an action to take based on that state and performing it. Each training lasted 2000 episodes, but the total number of interactions differs between training instances, because the duration of each episode is variable. As depicted in Table 5, all three *IA* variants lead to a decrease of the training time of over 30% compared to the *TF* methodology, with the case of $k_{b_2} = 4$ reaching the fastest optimization (improvement of 38.02%). This variant also produces more stable results between training samples, given the lower standard deviation of the training time (2.3312×10^3 interactions), and, thus, any further analysis concerning *IA* will consider $k_{b_2} = 4$. Fig. 6 presents the evolution of the cumulative rewards for three sampled training instances. The horizontal axis shows the number of interactions between the agent and the environment. Overall, the *IA* method led to an earlier convergence of the reward sum, that is, it required a lower number of interactions to reach a stable and successful policy.

Fig. 7 helps to explain this observation. It presents the number of episodes which terminated by means of the considered events: $\{\text{target, collision, turn_over, boundary}\}$, during the first 200 episodes out of a total of 2000. The depicted values represent the average computed over the ten training instances of each methodology and uncover the reason why the concept proposed in this article leads to a substantially

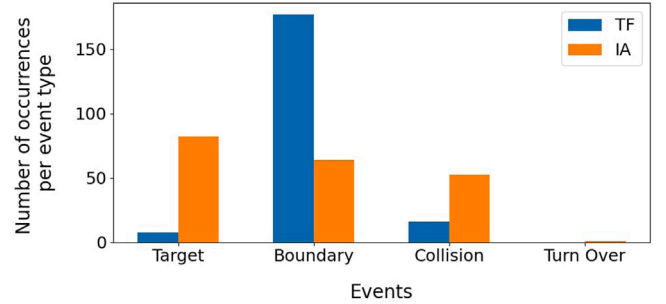


Fig. 7. Number of occurrences per event type during the first 200 episodes of the berthing training, considering an average computed across the ten training instances. The meaning of each event is: the vessel reaching its target position (Target); the vessel overcoming the dynamic boundary (Boundary); the vessel colliding with another object (Collision); or the vessel turning over (Turn Over).

lower training time. The *IA* method obtained a number of occurrences per event in the first 200 episodes that is more evenly distributed among the events, with the exception of *turn_over*. By blocking the vessel from taking contrary actions sequentially, the algorithm enforced its exploration and, as such, made the vessel experiment the different termination events early in the training process. This strategy contributed to a significant expansion of the agent's knowledge in the initial stage, which sped up the convergence of the cumulative rewards during the berthing training. On the other hand, the *TF* method did not restrict the agent's action selection, which implied its movement was not as broad and, thus, most of the initial episodes terminated through the shrinking boundary condition, as observed in Fig. 7. Given the conclusions drawn from this analysis, the docking algorithm was fully trained (from berthing to approach stage) using *IA* and, thus, the results presented throughout the rest of this section account for the implementation of this concept.

4.2.2. Episode analysis

The convergence of the cumulative rewards and the number of successful episodes do not determine *per se* the quality of the docking algorithm. Other features should be scrutinized, such as the vessel's general behaviour throughout each docking episode, the sequence of actions it takes and the evolution of its linear and angular velocities.

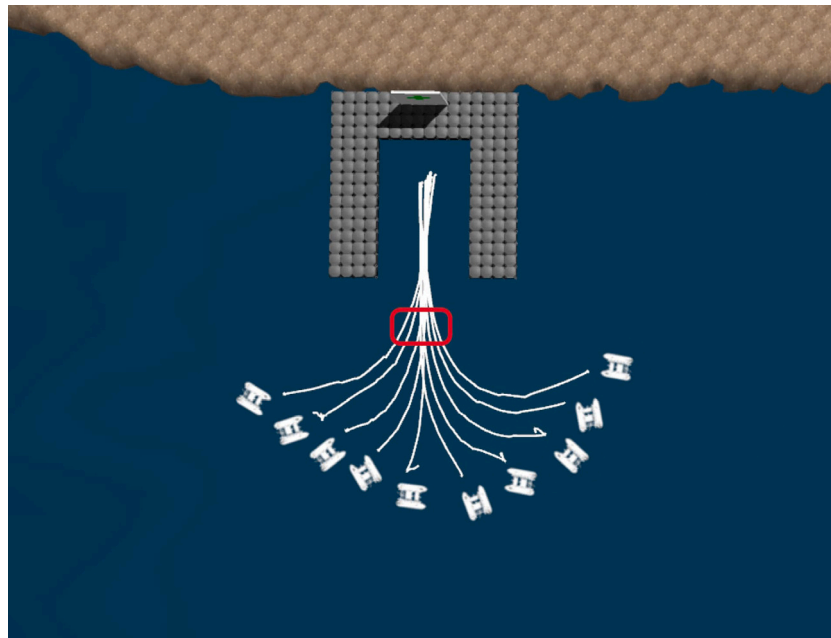


Fig. 8. Representation of the vessel's trajectory in 10 sample episodes. The agent firstly reaches an intermediate zone in front of the docking structure (depicted by a red rectangle) and then moves straight towards the target position.

Fig. 8 illustrates an interesting observation of the agent's behaviour. Considering no other obstacle in the scene besides the docking structure, the most efficient path for the vessel to take would be to move in a straight line to the dock's entry. Instead, the agent inherently learnt to reach an intermediate zone (depicted by a red rectangle in the figure), and then entering the structure already with the desired heading and quite centred within the passageway. This docking approach is not the optimal one in terms of minimizing the distance covered by the trajectory, but it does prevent collisions in a more effective manner and, thus, ensures a safer completion of the docking process. This strategy was not directly taught to the network, given that, unlike other works (Amendola et al., 2020), this work does not provide the agent with intermediate target poses, but only with the final pose within the structure. As such, the vessel learnt this behaviour through the collision risk assessment that it was provided with and through the consequences it received when interpreting this risk. Additionally, the IA methodology helped the agent learn the intermediate steps that lead to either collision or successful conclusion of the episode, and, thus, it constructed a strategy that avoids proximity to other obstacles as much as possible, especially during rotational movement.

The agent's behaviour throughout the docking maneuver can be discussed more in depth by analysing Fig. 9, which depicts a sampled testing episode. This particular episode lasted 126 interactions, which correspond to 25.2 s. On average, the tested docking episodes lasted 123 interactions (24.6 s). For this specific case, the evolution of the vessel's position and heading errors regarding the target pose, previously referred to as $dist$ and ψ , is depicted in Figs. 9(a) and 9(b), respectively. Both errors converge to approximately 0 m and 0 rad, although the heading error achieves this target earlier in the episode, which can be explained by the behaviour described above, in which the agent reaches an intermediate zone prior to entering the docking structure already with the desired heading. In fact, to understand how the agent accomplishes this, Figs. 9(c) to 9(f) illustrate the linear and angular velocities throughout the episode divided into the two main stages: approach (9(c) and 9(d)) and berthing (9(e) and 9(f)). In all four figures, the linear velocity is represented in blue, whereas the angular velocity is coloured in orange; the velocity command is represented by a solid line, whereas the actual velocity, constrained by the vehicle's dynamics and the environment, is depicted by a dashed line. The

approach stage initiates with the vessel rotating approximately towards the intermediate zone, moving in its direction and reorienting itself according to the target heading. As expected, the frequency of the commands increases as the vehicle gets closer to the structure. Fig. 9(e) demonstrates that the agent effectively learnt to use the two velocity levels (fast and slow) correctly, given that at first it chooses quicker movement and then switches to a slower approach as it draws near the target pose, in order not to collide with the docking structure. The chosen action sequence ensured that the vessel's velocity decreased gradually in the final interactions of the docking episode. A video of three docking instances was recorded in scenarios constructed in simulator Gazebo.²

4.2.3. Robustness to unknown conditions

As outlined throughout this work, one of the core objectives was to develop a framework with great capability of adaptation to unseen conditions. This section describes two different sets of experiments to demonstrate the robustness of the proposed research when performing in unknown environments: firstly, considering the effect of waves and wind, and secondly, incorporating different types of docking structures into the scenario according to the most common layouts found in commercial harbours.

Performance under environmental constraints. Environmental constraints predominant in maritime scenarios, such as wind and ocean waves, are still disregarded by many authors and, yet, highly affect navigation systems (Rees, 2021). This concern was not overlooked in this work, and Gazebo's features to simulate such conditions were put to use. Ocean waves are replicated by the sum of sinusoidal waves, whereas wind is reproduced by its x , y and yaw coefficients and speeds (Sarda et al., 2016). Each wave component is characterized by its amplitude, period and direction in the x and y axes.

The Beaufort Scale (World Meteorological Organization, 1970) classifies the intensity of wind based on its speed and on observed conditions at sea or on land, and the Douglas Sea Scale (Curto et al., 2021) measures the height of the waves. Due to its weight and dimensions, the

² <https://youtu.be/1IhD9AhRI-k>

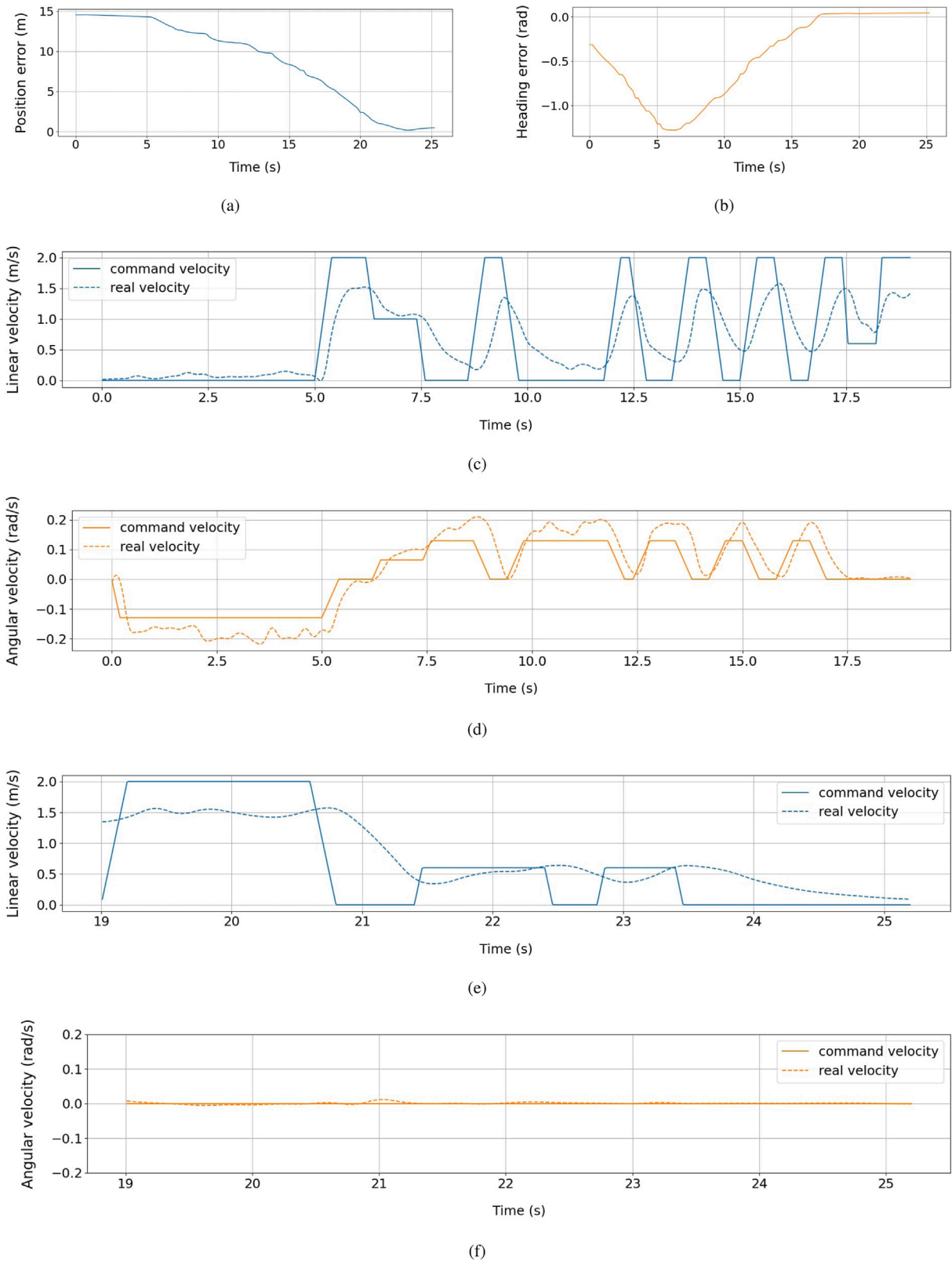


Fig. 9. Position (a) and heading (b) errors throughout the episode's interactions. Command and real velocities through the two docking stages: approach (c,d) and berthing (e,f).

SENSE ASV is able to operate under environmental constraints that correspond to levels 0 to 3 of both scales. Harsher conditions would cause this vessel to turn over. As such, this section presents some experiments conducted in scenarios that simulate the conditions of those four levels. Seven different scenarios were defined according to Table 6: Scn1 to

Scn7, where the former is the easiest setting and the latter is the most challenging. Each scenario is characterized by three wave components and one wind component. Fig. 10 illustrates two examples of wave components (10(a) and 10(b)), as well as their sum 10(c), which represents the actual wave that affects the vessel's movement. In this

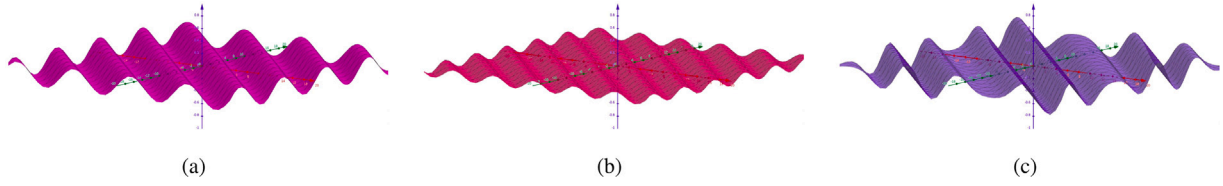


Fig. 10. Simulated wave components 1 (a) and 2 (b), as well as their sum (c). In this work, each scenario is characterized by three wave components.

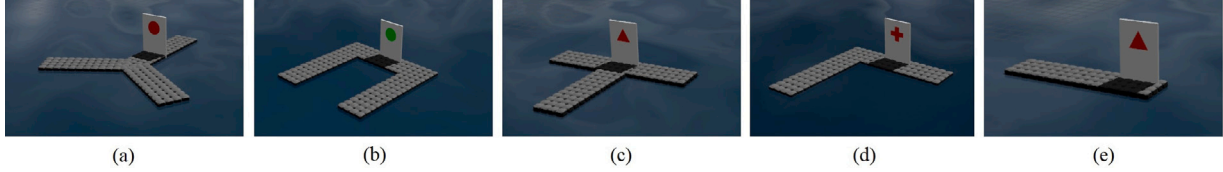


Fig. 11. Five of the most common dock models found in commercial harbours according to Pereira et al. (2021a). The illustrated layouts are Y-Shape (a), U-shape (b), T-shape (c), L-shape (d), and Straight (e).

Table 6

Scenario characterization according to the Beaufort Scale and the Douglas Sea Scale.

Scenario	Beaufort Scale	Douglas Sea Scale	Maximum wave amplitude (m)	Wind speed (m s^{-1})
Scn1	0	1	0.10	0.2
Scn2	1	1	0.17	0.5
Scn3	1	1	0.24	0.8
Scn4	1	1	0.29	1.4
Scn5	2	1	0.38	2.0
Scn6	2	2	0.50	2.6
Scn7	3	3	0.62	3.4

Table 7

Average results over 100 testing episodes per scenario (Scn1 to Scn7).

Scenario	Average position error (m)	Average heading error (rad)
Scn1	0.329	0.0241
Scn2	0.344	0.0240
Scn3	0.354	0.0295
Scn4	0.373	0.0401
Scn5	0.436	0.0464
Scn6	0.485	0.0524
Scn7	0.542	0.0544

experiment, the first wave component has the greatest amplitude and also the greatest period (7 s). The third component simulates the ocean ripple and therefore has a higher frequency (period of 2 s) but also a lower amplitude. The middle component has intermediate amplitude and period (5 s). The direction of the waves and wind elements were varied within each scenario according to: $(\vec{x}, \vec{y}) = (\lambda_x \cdot 0.7, \lambda_y \cdot 0.7)$, where λ_x and λ_y can either be -1 or 1 .

Each scenario was tested for 100 docking episodes in the simulator *Gazebo*. Table 7 presents the position and heading errors at the end of the testing episodes, averaged per scenario. All seven cases achieved a success rate of 100%. However, the positional error at the end of the docking maneuver increases with the aggravation of the scenario's constraints, with a minimum error of 0.329 m for Scn1 and a maximum one of 0.542 m for Scn7. The average heading error follows a similar tendency, varying between 0.0240 rad and 0.0544 rad. The harshness of the constraints portrayed in these simulations greatly increases between the first and last scenarios, but the position and heading errors only expand by 1.65 and 2.27 times, respectively. Considering maximum allowed position and heading errors of 1.0 m and 0.2 rad for achieving a successful docking episode, the worst case scenario of these experiments obtained errors of approximately half of those parameters' thresholds. Therefore, these results demonstrate that the agent was able to complete the maneuver within the safety parameters and with a conservative margin through all the scenario types.

Generalization to commercial docking structures. A robust docking system should retain its accuracy when operating in different scenarios and with different berthing structures, even if these were not seen during the algorithm's development stage. This was one of the motives for selecting an architecture based on Reinforcement Learning, since the knowledge gathered by the agent during the training stage can be generalized to diverse environments and does not need manual fine-tuning adapted to each new scenario. According to previous studies (Pereira et al., 2021a), there are five main docking structure layouts within commercial harbours, which are illustrated in Fig. 11. The layouts in the figure are represented by *Gazebo* models that were used for simulating docking scenarios different from the one adopted so far which used only a "U-shape" dock, depicted in Fig. 11(b). Unlike the U layout, some of the other dock types allow for more than one type of docking maneuver, e.g., frontward or side-ways approach, which further adds complexity to the task.

A study was conducted to measure the system's performance when dealing with these unseen docking structures. A total of 1000 episodes evenly distributed across the five types of docking layouts were tested in *Gazebo* and led to an average success rate of 99.9%, with one collision occurrence for dock type Y, which is the most complex layout and the least commonly found in harbours. A sample of these testing episodes were recorded in video.³

4.2.4. Comparison to conventional methods

As one of the first RL-based approaches towards autonomous docking, there are not many baselines available for performance comparison. A cautious analysis can be conducted using traditional methods that share some of the objectives of the proposed methodology, even if the core concepts significantly differ. Given the scope of this work, which focuses on intelligent decision-making rather than on enforcing specific low-level behaviour, this comparative experiment should be conducted among motion planning methodologies. For example, A* is a path-searching method known for its efficiency and completeness (Hart et al., 1968). It aims to find the shortest path between any two points on a grid-based map that includes obstacle information. Similarly to RL-based approaches, A* can be used to tackle high-level autonomous navigation with obstacle avoidance, although it simply provides target waypoints to be followed by the agent, whereas the proposed method constitutes an end-to-end framework, from state to action. In this analysis, the path computed by the A* algorithm is enforced on the agent using a simple PID controller. Moreover, the A*-based method requires careful parameter tuning to be deployed in different scenarios

³ <https://youtu.be/1%5fFzU-fnyvY>

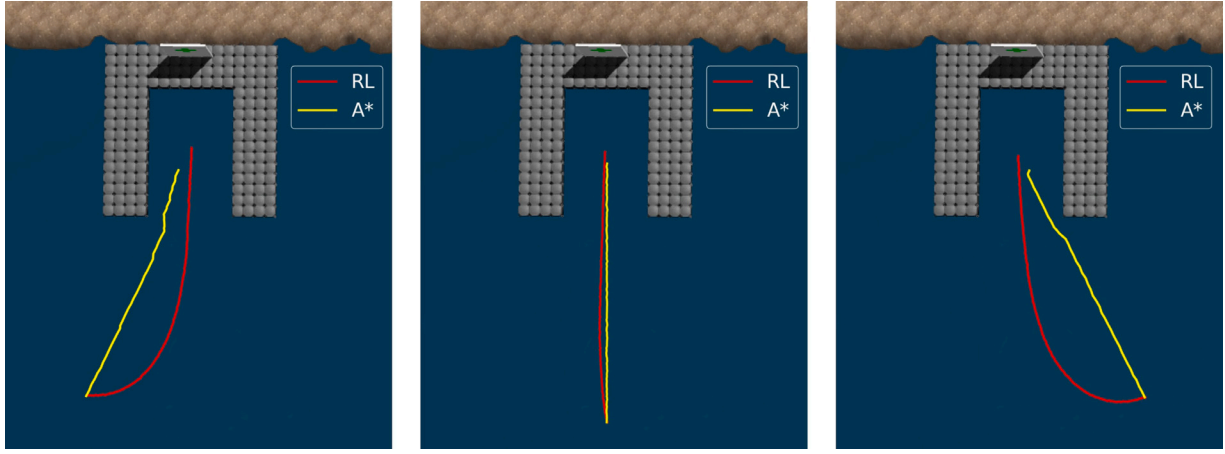


Fig. 12. Representation of three pairs of trajectories obtained by the A* (yellow) and the RL-based (red) methods. A* finds the shortest path to the target point, but it leads the agent closer to obstacles. RL takes a longer but safer trajectory.

or with different agents, whereas the approach proposed in this work can be readily used in unknown setups.

Considering this interpretation, a ROS-based implementation of A* was adapted to the problem at hand, using a grid-based map with a resolution of 0.15 m and an obstacle inflation radius equal to the length of the vehicle's model (1.50 m). Both the A* and RL-based methods were tested for 100 docking episodes in the simulator *Gazebo*, with initial poses for the agent being randomly selected according to the ranges presented in Section 4.1. Both methods achieved a success rate of 100%, and Fig. 12 depicts a few of the resulting trajectories. Performance was analysed according to episode duration and collision risk. The latter criterion is given by computing the average of the maximum collision risk parameter in any direction throughout the testing episodes, and it varies between 0 and 1. The RL-based approach obtained an average episode duration of 182.3 interactions (36.5 s), whereas A* achieved 176.3 interactions (35.3 s). The average risk parameter was 0.26 and 0.47, for RL and A*, respectively. The more traditional approach managed to find the shortest path to the target point, as depicted in Fig. 12, which does not necessarily correspond to a considerably lower episode duration given the sharper changes of direction required to complete the path. On the other hand, the RL-based algorithm produced a longer trajectory but a safer one, given the behaviour discussed in the previous subsection, in which the agent reaches an intermediate zone before entering the berthing phase to avoid proximity to other obstacles. The benefit of this trend is confirmed by the significant difference between the average collision risk of the two methods. The A* algorithm obtained a risk parameter 1.8 times greater because its resulting paths lead to a much greater closeness to the docking structure. This divergence between the trajectories produced by the two methods can be explained by the fact that the A* algorithm applies to generic contexts, whereas the development of the RL framework took into account the limitations faced specifically by the majority of maritime vehicles. While learning, the RL agent had to deal with the lack of degrees of freedom to which differential vessels are subjected to, and as such it grasped a way of guiding the ASV towards its target while avoiding maneuvers which are difficult for this type of vehicle to perform, especially in close range to structures.

Moreover, to demonstrate the superior resilience of the RL-based method to unseen conditions when compared to the more traditional approach, further experiments were conducted under the environmental constraints portrayed by the scenarios presented in the previous section. Table 8 presents the success rate and the average collision risk for the two methods when performing under scenarios Scn1 to Scn7. As explained previously, the proposed framework is able to maintain a success rate of 100% for all degrees of environmental constraints. Predictably, the average collision risk gradually increases: from 0.26 to

Table 8

Success rate (0 to 100%) and average collision risk (0 to 1) over 100 testing episodes per scenario (Scn1 to Scn7) and per method (RL vs. A*).

Scenario	Success rate		Average collision risk	
	RL	A*	RL	A*
Scn1	100%	100%	0.26	0.47
Scn2	100%	100%	0.28	0.48
Scn3	100%	100%	0.27	0.51
Scn4	100%	96%	0.29	0.55
Scn5	100%	91%	0.29	0.58
Scn6	100%	89%	0.31	0.60
Scn7	100%	83%	0.32	0.63

0.32 for the harshest scenario. These results remain considerably below the outcome of the A* method in the easiest setting, which obtained an average risk of 0.47. The increase in severity of the wind and ocean waves produces a more drastic result for this conventional approach. The more unreliable trajectories seen above lead to some collision occurrences when the vehicle is subject to greater external disturbance. As such, the success rate drops from 100% in the fourth scenario and decreases to 83% for Scn7. This lower performance is reflected in the greater average collision risk, which is approximately double that of the RL method in the harshest scenario.

The presented work can also be evaluated against other methods regarding the computational time required to process the perception information to obtain a decision for the agent at each iteration. For the proposed framework, this metric includes any point cloud processing, collision risk assessment, and the network inference time, whereas for the A*-based implementation, the map update and path generation are taken into account. The results of this experiment are depicted in Table 9, and demonstrate the superiority of the RL-based method, which achieved an average processing time of 6 ms, whereas the more conventional approach required approximately 15 ms, considering a map resolution of 0.15 m, as indicated in the beginning of the section. Different resolution values were also experimented with, to approximate the A*-based method to a continuous scenario, as considered in the RL case. As portrayed in the table, lower resolution values, which correspond to more accurate representations of the environment, lead to considerably higher processing times, but did not achieve the low collision risk produced by RL. On the other hand, greater resolution values decrease inference time at the expense of riskier trajectories. The A* method would take further effort of parameter tuning to reach a more conservative obstacle avoidance component and there would still be no guarantee of the resulting path being the most appropriate for maritime vehicles or of the inference time standing within a reasonable interval to enable autonomous navigation.

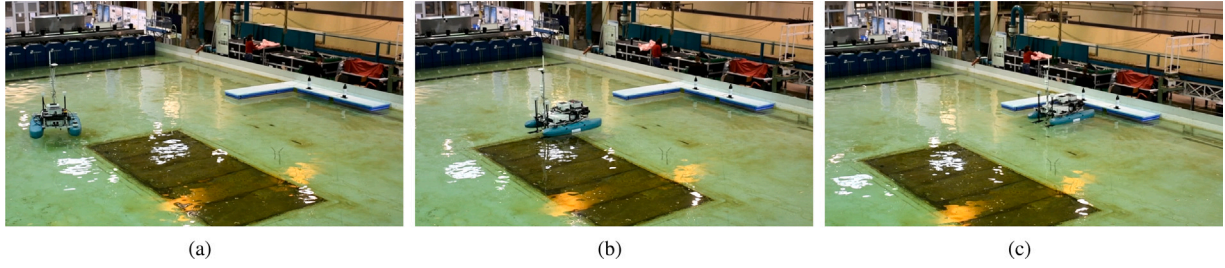


Fig. 13. The SENSE ASV during a docking episode in the water tank of the Civil Engineering Department of the Faculty of Engineering of the University of Porto, Portugal. The scenario features an L-shaped dock with a reflective beacon localization system. Frames (a), (b), and (c) depict the beginning, middle and end of the episode, respectively.

Table 9

Inference time (ms) and average collision risk (0 to 1) considering a continuous state representation for RL, and different grid map resolutions for A*.

	RL (continuous state)	A* Grid map resolution (m)				
		0.05	0.10	0.15	0.20	0.25
Inference time (ms)	6	101	64	15	13	9
Average collision risk	0.26	0.31	0.41	0.47	0.52	0.54

4.3. Validation in controlled scenario

One of this research's objectives was to advance the transition from the scientific domain to the technological one for autonomous docking, which could only be obtained through validation outside simulated environments. A set of hardware-in-the-loop trials enables the evaluation of the developed framework considering the constraints of real vehicles. Fig. 13 depicts the SENSE ASV within the indoor tank of the Civil Engineering Department of the Faculty of Engineering of the University of Porto, Portugal, where this set of experiments took place. Given that this is a GPS-denied environment, the docking framework relied on a reflective beacon localization system that used LiDAR point clouds to obtain the navigation-based features of the state space. This beacon system was only used for localization purposes, not for detecting the docking structure. As illustrated in the figure, the testing scenario featured an L-shaped dock made from extruded polystyrene foam (XPS) with dimensions $2.60 \times 3.30 \times 0.20$ m.

This experiment consisted of a total of 28 testing episodes, in which the ASV began its maneuver from randomly selected poses. The initial distance between the vehicle and the target varied between 3.049 m and 5.486 m, and its initial heading ranged between -1.8306 rad and 0.5796 rad. The criteria for episode termination are those presented in Table 4, as previously described, including a distance threshold (ϵ_{dist}) of 1.0 m and a heading threshold (ϵ_{θ}) of 0.2 rad. In this experiment, 27 episodes resulted in successful docking, which means the developed framework achieved a success rate of 96.4%. The average position and heading errors were of 0.965 m and 0.1732 rad, respectively, with maximum values of 0.997 m and 0.1997 rad, and minimum values of 0.084 m and 0.0163 rad. An average distance between the vehicle and the target of over 0.9 m constitutes an increase when compared to the tests presented in previous sections, but it actually represents the result of safer maneuvers. The low value of the maximum velocity adopted in these experiments enabled the vehicle to reduce its speed earlier and, therefore, stop before the target, thus decreasing the probability of colliding with the docking structure. In this experiment, a single episode failed to terminate successfully. During this attempt, the vessel was able to reach the target within the distance threshold but it could not achieve the desired heading within the episode's maximum duration.

Fig. 14 depicts a few of the successful trajectories to reach the docking target. The red circumference of radius equal to the distance threshold (ϵ_{dist}) represents the range within which the vehicle should

be positioned for the episode to terminate successfully. The red/green xy referential at the centre of the circumference depicts the ideal final pose, while the frame at the end of each path represents the vessel's actual orientation at the end of the episode. As previously explained, the majority of the docking episodes terminate before the vessel reaches the docking target, which results in a higher position error but in a lower collision risk with the docking structure. The path coloured in light brown represents 1 out of 3 docking instances in which the vehicle did not dock at first attempt, but instead it reached the structure with an offset in the y axis relative to the target spot and then moved backward a little to adjust its position and reach the distance threshold. This behaviour was not explicitly taught during the training process but was inherently learnt by the network after failed attempts. Most of the depicted trajectories present the behaviour described in Section 4.2.2, in which the vessel does not move in a straight line to the target, but reaches an intermediate zone first and then moves forward to the structure, for increased safety. The trajectories that do not conform to this behaviour, such as the blue and brown ones, departed from poses that were not considered as starting points during the training process (due to the utilized U-shaped docking structure). Nonetheless, even in these cases the vessel was able to reach the target within the safety parameters. On average, the 27 episodes lasted 118 interactions each (23.6 s).

A sample of the described experiment was recorded in video,⁴ along with a brief explanation of the vessel's behaviour throughout the docking maneuver.

4.4. Validation in real harbour

The final validation scenario took place in the ATLANTIS Test Centre (Pinto et al., 2021), in Viana do Castelo, Portugal. This is a testing platform for maritime O&M robotics, that includes a coastal testbed which features a floating foundation from an offshore wind turbine, as well as multiple docking structures. As previously mentioned, the vessel used in these tests was the Nautilus ASV (Campos et al., 2024), developed by INESC TEC, which is depicted in Fig. 15 during an operation at the test centre. Throughout the experiments, the registered environmental conditions were the following: wind of $[4,5] \text{ m s}^{-1}$; wind gusts of $[7,9] \text{ m s}^{-1}$; and waves of two swell components (swell 1 - amplitude of 0.9 m and period of 6 s; swell 2 - amplitude of 0.2 m and period of 10 s). This set of conditions reach level 5 of the Beaufort Scale and the upper end of level 3 of the Douglas Sea Scale, and thus represent a scenario which is harsher than the most severe scenario portrayed in Section 4.2.3 - Robustness to Environmental Constraints.

In this experiment, ten successful docking episodes were achieved, in which the agent's initial distance to the target varied between 12.705 m and 21.806 m, and its initial heading error extended from -1.507 rad to 0.858 rad. Regarding the criteria for episode termination, the linear and angular velocity thresholds, $\epsilon_{v_{lin}}$ and $\epsilon_{v_{ang}}$, were loosened to 0.4 m s^{-1} and 0.1 rad s^{-1} , respectively, due to the strong current at

⁴ <https://youtu.be/enZokPffH-bU>

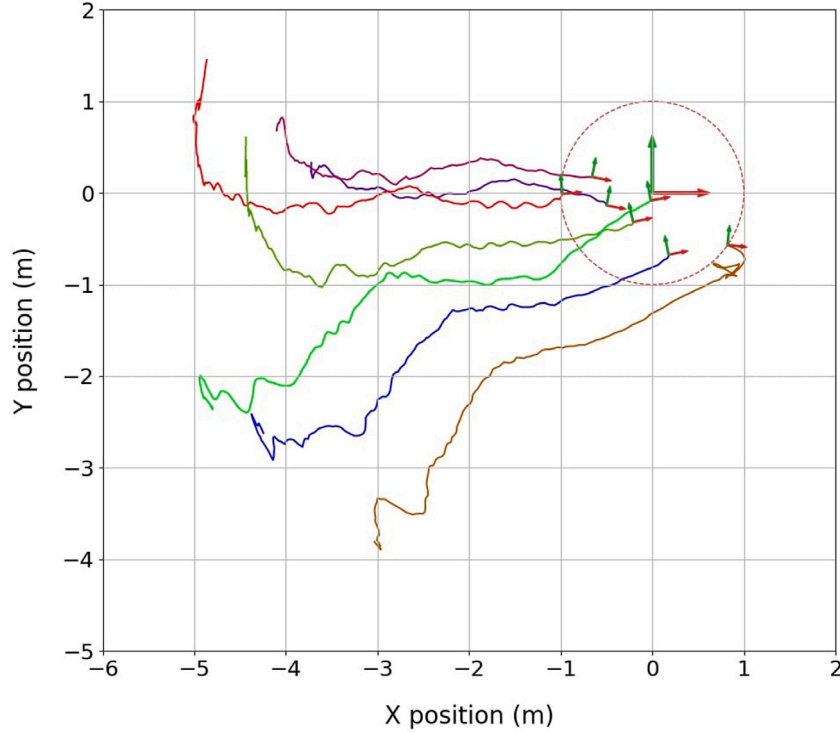


Fig. 14. Representation of the vessel's trajectory in 7 sample episodes. The red circumference of radius equal to 1.0 m (ϵ_{dist}) depicts the range within which the vehicle is considered to be correctly positioned. The xy frame in the centre of the circumference corresponds to the target pose, and the frame at the end of each trajectory represents the final orientation of the vehicle.



Fig. 15. The Nautilus ASV operating in the ATLANTIS Test Centre, in Viana do Castelo, Portugal.

the time of testing. The distance (ϵ_{dist}) and heading (ϵ_{θ}) thresholds were kept at 1.0 m and 0.2 rad, respectively, according to the training procedure and the previous testing scenarios. Over the ten docking samples, the framework obtained average position and heading errors of 0.841 m and 0.117 rad, which can be placed well within the corresponding thresholds. An average episode lasted 177.3 interactions, i.e., approximately 35.5 s.

Fig. 16 depicts four of the ten docking episodes of this experiment. For each of the four episodes, the starting pose is represented by $P_i, i \in \{0, 1, 2, 3\}$, while the trajectories performed by the vehicle are depicted by yellow lines, and a visual model of the Nautilus ASV is

represented at the target pose. These episodes featured two different docking structures and varied initial poses. All trajectories seem to be slightly disrupted by the environmental constraints, in particular by the wave components (swell 1 and 2), given that the vessel's trajectory presents some overshoot when it moves perpendicularly to the waves direction. This idea is supported by Fig. 17, representative of the path beginning in pose P_2 , in which the left vertical axis shows the derivative of the vessel's heading ($\dot{\psi}$), and as such, relates to the trajectory's smoothness, while the right vertical axis presents the difference in heading ($\Delta\psi_{waves}$) between the vessel and the waves (given by the combination of the swell components). The chart demonstrates that

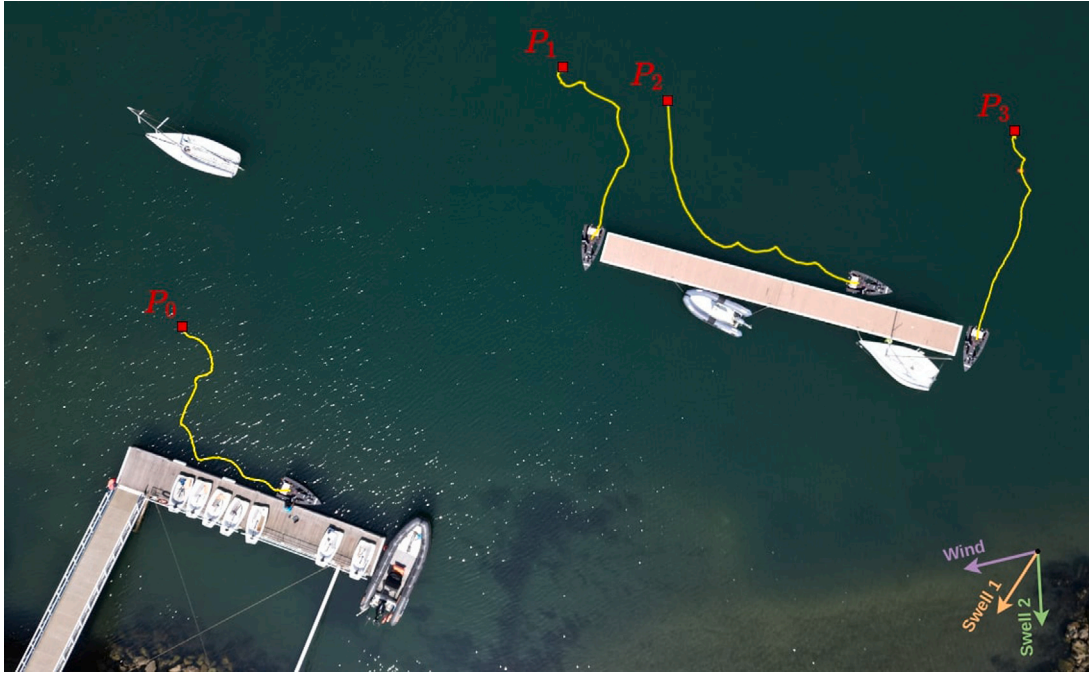


Fig. 16. Representation of Nautilus' trajectory (yellow) in four sample docking episodes in two different docking structures at the ATLANTIS Test Centre. For each of the four docking episodes, the starting pose is represented by $P_i, i \in \{0, 1, 2, 3\}$, and the ASV is depicted at the target pose. The direction of the wind and wave components (swell 1 and 2) is outlined in the bottom right corner of the figure.

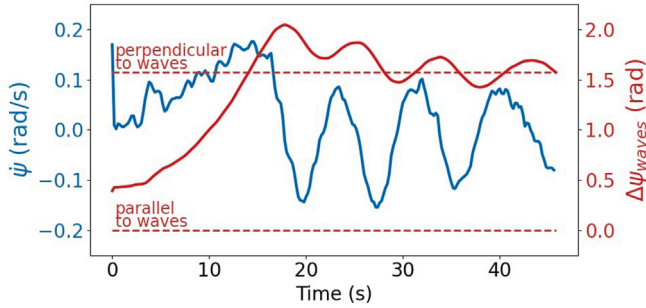


Fig. 17. Analysis of the trajectory beginning in pose P_2 . The left vertical axis shows the derivative of the vessel's heading (ψ), while the right vertical axis presents the difference in heading between the vessel and the waves ($\Delta\psi_{waves}$).

when the ASV is moving relatively parallel to the waves, then ψ remains fairly stable. The abrupt heading variation at the start of the episode corresponds to the initial reorientation of the vessel, when no translation takes place. On the other hand, when $\Delta\psi_{waves}$ draws closer to $\frac{\pi}{2}$ rad (≈ 1.571 rad), the heading derivative becomes more volatile, alternating between the positive and negative ranges. This instability results from the framework's response to counteract these environmental disturbances, not only to return to an optimal path towards the target but also to avoid collision with the docking structure which features in close range to the vessel during the second half of this particular episode. Such response is enabled by the framework's ability to adapt to unpredictable states, leveraged by the onboard perception system, and so these disruptions end up not hindering the success of the docking maneuver.

Three other sampled episodes were recorded in a video⁵ which better depicts the testing environment, along with the environmental constraints and the crowdedness of the harbour. In one of the docking

episodes presented in the video, the agent's action is seen to be disturbed by a set of larger waves, caused by the movement of a bigger vessel. However, the framework manages to redirect the agent to a converging trajectory that prevents collision to the structure, and the docking episode terminates with success. Despite being more challenging, this episode achieved position and heading errors of m0.857 and 0.015 rad, respectively, which scarcely deviate from the average results obtained during these tests. In a different episode, depicted in the sequential frames of Fig. 18, the agent is faced with an obstacle between its initial and target poses. The agent is able to deviate from said obstacle and conclude the docking maneuver, owing to its onboard perception system and the diversified framework training process.

To the best of the authors' knowledge, this is the first RL-based autonomous docking framework for ASVs to be tested in a real environment. The results presented in this work depict not only the potential of the methodology, but also its robustness and flexibility. These final validation tests covered greater distances than those portrayed in scenarios (i) and (ii), as well as harsher environmental conditions, which demonstrates the capability of the RL framework to adapt to different and more difficult backgrounds without retraining. This methodology's versatility is further evidenced by having been validated with two different ASVs (SENSE and Nautilus), with different sizes, configurations and velocities of operation. The use of different localization methods and the fact that all situational awareness is computed through the onboard perception system show that the docking framework can be readily deployed to unknown and complex scenarios.

5. Conclusion

This article proposed a framework for autonomous docking of surface vehicles in real environments through a Deep Reinforcement Learning agent. The concept of *illegal actions* was introduced to improve the agent's exploration during the learning process, by increasing the coherence of action selection from a high-level action space. This research also allowed to validate the potential of A2C-based architectures for critical decision-making problems, such as docking operations. Unlike most of the current methods, this work addressed some challenges

⁵ <https://youtu.be/DgsDWZ5WgIM>

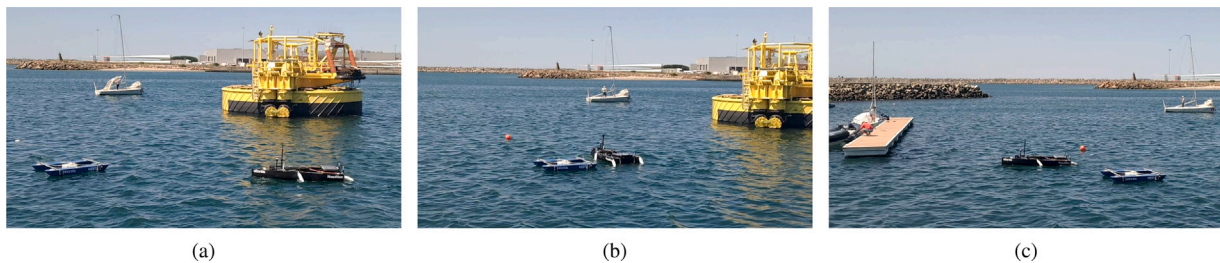


Fig. 18. The Nautilus ASV during a docking episode at the ATLANTIS Test Centre, in which the RL agent is able to deviate from an obstacle using information from the vessel's onboard perception system.

presented in real-world scenarios, such as environmental constraints (wind and waves) and the lack of prior knowledge of the scene. By computing all situational awareness using the vessel's onboard perception system, this framework is able to guide the agent into unknown scenarios.

The methodology based on the concept of *illegal actions* was thoroughly tested and demonstrated a reduction of 38.02% on the framework's optimization time, by focusing the training process on meaningful experiences met by the agent. A comprehensive analysis showed robustness to different environmental conditions, according to the Beaufort Scale (levels 0 to 5) and the Douglas Sea Scale (levels 1 to 3), and also considering different types of docking structures. Moreover, a comparative analysis with a state-of-the-art path planning method demonstrated the superior ability of the developed learning-based framework regarding safety, inference time and generalization. The developed framework was tested using a real ASV in a controlled scenario, having obtained a success rate of 96.4%. Validation in a real harbour with a different vehicle, and under more severe weather conditions, showed the framework's ability to maintain a solid performance, unlike other RL-based works that went through real-world deployment. In conclusion, the proposed framework optimized the training of an RL agent that resulted in a robust autonomous docking methodology which does not rely on *a priori* knowledge of the scenario and can be deployed to real-world setups.

Future research lines could involve experimenting in more complex scenarios, while using different types of surface vehicles, such as holonomic ones, to fully understand the generalization capabilities of the developed framework. Furthermore, the pipeline could be improved by simplifying the pre-processing stages and providing rawer perception data to the network so that the system becomes even less dependent on specific scenarios. Lastly, the proposed framework could be integrated with a docking structures detection system to achieve an end-to-end operation.

CRediT authorship contribution statement

Maria Inês Pereira: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Andry Maykol Pinto:** Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

This work is partly funded by the Portuguese Government through the FCT - Foundation for Science and Technology within the Ph.D. studentship 2021.05561.BD (to MP), and by the European Union's Horizon 2020 - The EU Framework Programme for Research and Innovation 2014–2020, under grant agreement No. 871571 (ATLANTIS).

References

- Alexandru Zamfirache, I., Precup, R.-E., Roman, R.-C., Petriu, E.M., 2023. Neural network-based control using actor-critic reinforcement learning and grey wolf optimizer with experimental servo system validation. *Expert Syst. Appl.* 225, 120112. <http://dx.doi.org/10.1016/j.eswa.2023.120112>.
- Amendola, J., Miura, L.S., Costa, A.H.R., Cozman, F.G., Tannuri, E.A., 2020. Navigation in restricted channels under environmental conditions: Fast-time simulation by asynchronous deep reinforcement learning. *IEEE Access* 8, 149199–149213. <http://dx.doi.org/10.1109/ACCESS.2020.3015661>.
- Anderlini, E., Parker, G.G., Thomas, G., 2019. Docking control of an autonomous underwater vehicle using reinforcement learning. *Appl. Sci.* 9 (17), <http://dx.doi.org/10.3390/app9173456>.
- Bartels, S., Helling, S., Meurer, T., 2022. Rope-assisted docking maneuvers for autonomous surface vessels. In: 2022 Am. Cont. Conf., ACC, pp. 2315–2320. <http://dx.doi.org/10.23919/ACC53348.2022.9867315>.
- Bjering Strand, H., 2020. Autonomous Docking Control System for the Otter USV: A Machine Learning Approach. (Master's thesis). Norwegian University of Science and Technology (NTNU).
- Cai, W., Zhang, M., Yang, Q., Wang, C., Shi, J., 2023. Long-range UWB positioning-based automatic docking trajectory design for unmanned surface vehicle. *IEEE Trans. Instrum. Meas.* 72, 1–12. <http://dx.doi.org/10.1109/TIM.2023.3271005>.
- Campos, D.F., Gonçalves, E.P., Campos, H.J., Pereira, M.I., Pinto, A.M., 2024. Nautilus: An autonomous surface vehicle with a multilayer software architecture for offshore inspection. *J. Field Rob.* <http://dx.doi.org/10.1002/rob.22304>.
- Campos, D.F., Matos, A., Pinto, A.M., 2019. An adaptive velocity obstacle avoidance algorithm for autonomous surface vehicles. In: *Proc. - 2019 IEEE/RSJ Int. Conf. Intell. Robot. Syst., IROS, Macau, China*, pp. 8089–8096. <http://dx.doi.org/10.1109/IROS40897.2019.8968156>.
- Campos, D., Matos, A., Pinto, A., 2021a. Multi-domain inspection of offshore wind farms using an autonomous surface vehicle. *SN Appl. Sci.* 3 (455), <http://dx.doi.org/10.1007/s42452-021-04451-5>.
- Campos, D.F., Pereira, M., Matos, A., Pinto, A.M., 2021b. DIIUS - distributed perception for inspection of aquatic structures. In: *OCEANS 2021: San Diego – Porto*, pp. 1–5. <http://dx.doi.org/10.23919/OCEANS44145.2021.9705939>.
- Cao, X., Sun, C., Yan, M., 2019. Target search control of AUV in underwater environment with deep reinforcement learning. *IEEE Access* 7, 96549–96559. <http://dx.doi.org/10.1109/ACCESS.2019.2929120>.
- Capra, L., Brandonisio, A., Lavagna, M., 2023. Network architecture and action space analysis for deep reinforcement learning towards spacecraft autonomous guidance. *Adv. Space Res.* 71 (9), 3787–3802. <http://dx.doi.org/10.1016/j.asr.2022.11.048>.
- Cebollada, S., Payá, L., Flores, M., Peidró, A., Reinoso, O., 2021. A state-of-the-art review on mobile robotics tasks using artificial intelligence and visual data. *Expert Syst. Appl.* 167, 114195. <http://dx.doi.org/10.1016/j.eswa.2020.114195>.
- Chang, L., Shan, L., Jiang, C., Dai, Y., 2021. Reinforcement based mobile robot path planning with improved dynamic window approach in unknown environment. *Auton. Rob.* 45, <http://dx.doi.org/10.1007/s10514-020-09947-4>.
- Chu, Z., Wang, F., Lei, T., Luo, C., 2023. Path planning based on deep reinforcement learning for autonomous underwater vehicles under ocean current disturbance. *IEEE Trans. Intell. Veh.* 8 (1), 108–120. <http://dx.doi.org/10.1109/ITV.2022.3153352>.
- Chun, D.-H., Roh, M.-I., Lee, H.-W., Ha, J., Yu, D., 2021. Deep reinforcement learning-based collision avoidance for an autonomous ship. *Ocean Eng.* 234, 109216. <http://dx.doi.org/10.1016/j.oceaneng.2021.109216>.

- Claro, R., Silva, R., Pinto, A., 2020. Detection and mapping of monopiles in offshore wind farms using autonomous surface vehicles. In: *Global Oceans 2020: Singapore – U.S. Gulf Coast*. pp. 1–4. <http://dx.doi.org/10.1109/IEEECONF38699.2020.9389348>.
- Curto, D., Franzitta, V., Guercio, A., 2021. Sea wave energy. a review of the current technologies and perspectives. *Energies* 14 (20), <http://dx.doi.org/10.3390/en14206604>.
- Dann, M., Thangarajah, J., 2021. Adapting to reward progressivity via spectral reinforcement learning. <http://dx.doi.org/10.48550/arXiv.2104.14138>, arXiv:2104.14138.
- Darbra, R.M., Casal, J., 2004. Historical analysis of accidents in seaports. *Saf. Sci.* 42 (2), 85–98. [http://dx.doi.org/10.1016/S0925-7535\(03\)00002-X](http://dx.doi.org/10.1016/S0925-7535(03)00002-X).
- de Bruin, T., Kober, J., Tuyls, K., Babuška, R., 2018. Integrating state representation learning into deep reinforcement learning. *IEEE Rob. Autom. Lett.* 3 (3), 1394–1401. <http://dx.doi.org/10.1109/LRA.2018.2800101>.
- Duguleana, M., Mogan, G., 2016. Neural networks based reinforcement learning for mobile robots obstacle avoidance. *Expert Syst. Appl.* 62, 104–115. <http://dx.doi.org/10.1016/j.eswa.2016.06.021>.
- Garaffa, L.C., Basso, M., Konzen, A.A., de Freitas, E.P., 2021. Reinforcement learning for mobile robotics exploration: A survey. *IEEE Trans. Neural Netw. Learn. Syst.* 1–15. <http://dx.doi.org/10.1109/TNNLS.2021.3124466>.
- Gjærum, V.B., Rørvik, E.-L.H., Lekkas, A.M., 2021. Approximating a deep reinforcement learning docking agent using linear model trees. In: *2021 European Control Conference. ECC*, pp. 1465–1471. <http://dx.doi.org/10.23919/ECC54610.2021.9655007>.
- Grondman, I., Busoniu, L., Lopes, G.A.D., Babuska, R., 2012. A survey of actor-critic reinforcement learning: Standard and natural policy gradients. *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.* 42 (6), 1291–1307. <http://dx.doi.org/10.1109/TSMCC.2012.2218595>.
- Hadi, B., Khosravi, A., Sarhadi, P., 2024. Adaptive formation motion planning and control of autonomous underwater vehicles using deep reinforcement learning. *IEEE J. Oceanic Eng.* 49 (1), 311–328. <http://dx.doi.org/10.1109/JOE.2023.3278290>.
- Hameed, M.S.A., Chadha, G.S., Schwung, A., Ding, S.X., 2021. Gradient monitored reinforcement learning. *IEEE Trans. Neural Netw. Learn. Syst.* 1–14. <http://dx.doi.org/10.1109/TNNLS.2021.3119853>.
- Hart, P.E., Nilsson, N.J., Raphael, B., 1968. A formal basis for the heuristic determination of minimum cost paths. *EEE Trans. Syst. Sci. Cybern.* 4 (2), 100–107. <http://dx.doi.org/10.1109/TSSC.1968.300136>.
- Higo, Y., Sakano, M., Nobe, H., Hashimoto, H., 2023. Development of trajectory-tracking maneuvering system for automatic berthing/unberthing based on double deep Q-network and experimental validation with an actual large ferry. *Ocean Eng.* 287, 115750. <http://dx.doi.org/10.1016/j.oceaneng.2023.115750>.
- Holen, M., Ruud, E.-L.M., Warakagoda, N.D., Goodwin, M., Engelstad, P., Knausgård, K.M., 2022. Towards using reinforcement learning for autonomous docking of unmanned surface vehicles. In: *Eng. Appl. Neural Networks*. Springer International Publishing, Cham, pp. 461–474. http://dx.doi.org/10.1007/978-3-031-08223-8_38.
- Hu, Y., Wang, W., Jia, H., Wang, Y., Chen, Y., Hao, J., Wu, F., Fan, C., 2020. Learning to utilize shaping rewards: A new approach of reward shaping. In: *34th Conf. Neural Inf. Proc. Syst. (NeurIPS 2020)*. Vol. 33, pp. 15931–15941.
- Kim, H., Kim, D., Park, B., Lee, S.-M., 2020. Artificial intelligence vision-based monitoring system for ship berthing. *IEEE Access* 8, 227014–227023. <http://dx.doi.org/10.1109/ACCESS.2020.3045487>.
- Lambert, R., Li, J., Wu, L.-F., Mahmoudian, N., 2021. Robust ASV navigation through ground to water cross-domain deep reinforcement learning. *Front. Rob. AI* 8, <http://dx.doi.org/10.3389/frobt.2021.739023>.
- Leite, P.N., Pinto, A.M., 2024. Fusing heterogeneous tri-dimensional information for reconstructing submerged structures in harsh sub-sea environments. *Inf. Fusion* 103, 102126. <http://dx.doi.org/10.1016/j.inffus.2023.102126>.
- Leite, A., Pinto, A., Matos, A., 2018. A safety monitoring model for a faulty mobile robot. *Rob. 7*, 32. <http://dx.doi.org/10.3390/robotics7030032>.
- Leite, P., Silva, R., Matos, A., Pinto, A.M., 2019. An hierarchical architecture for docking autonomous surface vehicles. In: *Proc. - 19th IEEE Int. Conf. Auton. Robot Syst. Compet. ICARSC*, pp. 1–6. <http://dx.doi.org/10.1109/ICARSC.2019.8733620>.
- Li, S., Liu, J., Negenborn, R.R., Wu, Q., 2020. Automatic docking for underactuated ships based on multi-objective nonlinear model predictive control. *IEEE Access* 8, 70044–70057. <http://dx.doi.org/10.1109/ACCESS.2020.2984812>.
- Lin, X., McConnell, J., Englot, B., 2023. Robust unmanned surface vehicle navigation with distributional reinforcement learning. In: *Proc. - 2023 IEEE/RSJ Int. Conf. Intell. Robot. Syst. IROS*, pp. 6185–6191. <http://dx.doi.org/10.1109/IROS55552.2023.10342389>.
- Liu, H., Peng, Z., Gu, N., Wang, H., Liu, L., Wang, D., 2024. Collision-free automatic berthing of maritime autonomous surface ships via safety-certified active disturbance rejection control. *ISA Trans.* <http://dx.doi.org/10.1016/j.isatra.2024.03.010>.
- Løver, J., Gjærum, V.B., Lekkas, A.M., 2021. Explainable AI methods on a deep reinforcement learning agent for automatic docking. *IFAC-PapersOnLine* 54 (16), 146–152. <http://dx.doi.org/10.1016/j.ifacol.2021.10.086>.
- Martinsen, A.B., Bitar, G., Lekkas, A.M., Gros, S., 2020. Optimization-based automatic docking and berthing of ASVs using exteroceptive sensors: Theory and experiments. *IEEE Access* 8, 204974–204986. <http://dx.doi.org/10.1109/ACCESS.2020.3037171>.
- Meyer, E., Heiberg, A., Rasheed, A., San, O., 2020a. COLREG-compliant collision avoidance for unmanned surface vehicle using deep reinforcement learning. *IEEE Access* 8, 165344–165364. <http://dx.doi.org/10.1109/ACCESS.2020.3022600>.
- Meyer, E., Robinson, H., Rasheed, A., San, O., 2020b. Taming an autonomous surface vehicle for path following and collision avoidance using deep reinforcement learning. *IEEE Access* 8, 41466–41481. <http://dx.doi.org/10.1109/ACCESS.2020.2976586>.
- Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning. In: *Proc. 33rd Int. Conf. Machine Learning*. Vol. 48, New York, USA, pp. 1928–1937.
- Nguyen, V.S., 2019. Research on a support system for automatic ship navigation in fairway. *Future Internet* 11 (2), 38. <http://dx.doi.org/10.3390/fi11020038>.
- Pereira, M.I., Claro, R.M., Leite, P.N., Pinto, A.M., 2021a. Advancing autonomous surface vehicles: A 3D perception system for the recognition and assessment of docking-based structures. *IEEE Access* 9, 53030–53045. <http://dx.doi.org/10.1109/ACCESS.2021.3070694>.
- Pereira, M.I., Leite, P., Pinto, A., 2021b. A 3-D lightweight convolutional neural network for detecting docking structures in cluttered environments. *Mar. Technol. Soc. J.* 55, 88–98. <http://dx.doi.org/10.4031/MTSJ.55.4.9>.
- Pfeiffer, M., Shukla, S., Turchetta, M., Cadena, C., Krause, A., Siegwart, R., Nieto, J., 2018. Reinforced imitation: Sample efficient deep reinforcement learning for mapless navigation by leveraging prior demonstrations. *IEEE Rob. Autom. Lett.* 3 (4), 4423–4430. <http://dx.doi.org/10.1109/LRA.2018.2869644>.
- Pinto, A.M., Marques, J.V.A., Campos, D.F., Abreu, N., Matos, A., Jussi, M., Berglund, R., Halmé, J., Tikka, P., Formiga, J., Verrecchia, C., Langiano, S., Santos, C., Sá, N., Stoker, J.-J., Calderoni, F., Govindaraj, S., But, A., Gale, L., Ribas, D., Hurtós, N., Vidal, E., Rida, P., Chieslak, P., Palomeras, N., Barberis, S., Aceto, L., 2021. ATLANTIS - the atlantic testing platform for maritime robotics. In: *OCEANS 2021: San Diego – Porto*. pp. 1–5. <http://dx.doi.org/10.23919/OCEANS44145.2021.9706059>.
- Pinto, A.M., Matos, A.C., 2020. MARESys: A hybrid imaging system for underwater robotic applications. *Inf. Fusion* 55, 16–29. <http://dx.doi.org/10.1016/j.inffus.2019.07.014>.
- Rees, C., 2021. *A Master's Guide to Berthing*. The Standard Club, London, UK.
- Rigatos, G., Siano, P., Selisteanu, D., Precup, R.E., 2017. Nonlinear optimal control of oxygen and carbon dioxide levels in blood. *Intell. Ind. Syst.* 3 (2), 61–75. <http://dx.doi.org/10.1007/s40903-016-0060-y>.
- Roman, R.-C., Precup, R.-E., Hedrea, E.-L., Preitl, S., Zamfirache, I.A., Bojan-Dragos, C.-A., Petriu, E.M., 2022. Iterative feedback tuning algorithm for tower crane systems. *Procedia Comput. Sci.* 199, 157–165. <http://dx.doi.org/10.1016/j.procs.2022.01.020>.
- Rørvik, E.-L.H., 2020. *Automatic Docking of an Autonomous Surface Vessel*. (Master's thesis). Norwegian University of Science and Technology (NTNU).
- Sarda, E.I., Qu, H., Bertaska, I.R., von Ellenrieder, K.D., 2016. Station-keeping control of an unmanned surface vehicle exposed to current and wind disturbances. *Ocean Eng.* 127, 305–324. <http://dx.doi.org/10.1016/j.oceaneng.2016.09.037>.
- Schulman, J., Levine, S., Moritz, P., Jordan, M.I., Abbeel, P., 2015. Trust region policy optimization. <http://dx.doi.org/10.48550/arXiv.1502.05477>, arXiv:1502.05477.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. <http://dx.doi.org/10.48550/arXiv.1707.06347>, arXiv:1707.06347.
- Silva, R., Leite, P., Campos, D., Pinto, A.M., 2019. Hybrid approach to estimate a collision-free velocity for autonomous surface vehicles. In: *Proc. - 19th IEEE Int. Conf. Auton. Robot Syst. Compet. ICARSC*, Porto, Portugal, pp. 1–6. <http://dx.doi.org/10.1109/ICARSC.2019.8733643>.
- Sutton, R.S., Barto, A.G., 2017. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA.
- Valdez, F., Castillo, O., Cortes-Antonio, P., Melin, P., Kahraman, C., 2020. A survey of type-2 fuzzy logic controller design using nature inspired optimization. *J. Intell. Fuzzy Systems* 39 (5), 6169–6179. <http://dx.doi.org/10.3233/JIFS-189087>.
- Volden, Ø., Stahl, A., Fossen, T.I., 2022. Vision-based positioning system for auto-docking of unmanned surface vehicles (USVs). *Int. J. Intell. Rob. Appl.* 6 (1), 86–103. <http://dx.doi.org/10.1007/s41315-021-00193-0>.
- Volden, Ø., Stahl, A., Fossen, T.I., 2023. Development and experimental validation of visual-inertial navigation for auto-docking of unmanned surface vehicles. *IEEE Access* 11, 45688–45710. <http://dx.doi.org/10.1109/ACCESS.2023.3274597>.
- Wang, X., Deng, Z., Peng, H., Wang, L., Wang, Y., Tao, L., Lu, C., Peng, Z., 2023b. Autonomous docking trajectory optimization for unmanned surface vehicle: A hierarchical method. *Ocean Eng.* 279, 114156. <http://dx.doi.org/10.1016/j.oceaneng.2023.114156>.
- Wang, N., Gao, Y., Zhang, X., 2021a. Data-driven performance-prescribed reinforcement learning control of an unmanned surface vehicle. *IEEE Trans. Neural Netw. Learn. Syst.* 32 (12), 5456–5467. <http://dx.doi.org/10.1109/TNNLS.2021.3056444>.
- Wang, N., Gao, Y., Zhao, H., Ahn, C.K., 2021b. Reinforcement learning-based optimal tracking control of an unknown unmanned surface vehicle. *IEEE Trans. Neural Netw. Learn. Syst.* 32 (7), 3034–3045. <http://dx.doi.org/10.1109/TNNLS.2020.3009214>.
- Wang, P., Liu, R., Tian, X., Zhang, X., Qiao, L., Wang, Y., 2023a. Obstacle avoidance for environmentally-driven USVs based on deep reinforcement learning in large-scale uncertain environments. *Ocean Eng.* 270, 113670. <http://dx.doi.org/10.1016/j.oceaneng.2023.113670>.

- Wang, X., Wang, Y., Zhou, W., Zhang, J., 2023c. Pursuit-evasion game of unmanned surface vehicles based on deep reinforcement learning. In: 2023 4th Int. Conf. Electron. Commun. and Artif. Intell.. ICECAI, pp. 358–363. <http://dx.doi.org/10.1109/ICECAI58670.2023.10176487>.
- Woo, J., Kim, N., 2020. Collision avoidance for an unmanned surface vehicle using deep reinforcement learning. Ocean Eng. 199, 107001. <http://dx.doi.org/10.1016/j.oceaneng.2020.107001>.
- World Meteorological Organization, 1970. The Beaufort Scale of Wind Force : (Technical and Operational Aspects). WMO, Geneva, URL: <https://search.library.wisc.edu/catalog/999768644902121>.
- Xia, J., Zhu, X., Liu, Z., Luo, Y., Wu, Z., Wu, Q., 2023. Research on collision avoidance algorithm of unmanned surface vehicle based on deep reinforcement learning. IEEE Sens. J. 23 (11), 11262–11273. <http://dx.doi.org/10.1109/JSEN.2022.3222575>.
- Zhang, Q., Pan, W., Reppa, V., 2020. Model-reference reinforcement learning control of autonomous surface vehicles. In: 2020 59th IEEE Conf. Decis. Control. CDC, pp. 5291–5296. <http://dx.doi.org/10.1109/CDC42340.2020.9304347>.
- Zhang, Q., Pan, W., Reppa, V., 2022. Model-reference reinforcement learning for collision-free tracking control of autonomous surface vehicles. IEEE Trans. Intell. Transp. Syst. 23 (7), 8770–8781. <http://dx.doi.org/10.1109/TITS.2021.3086033>.
- Zhang, M., Yu, S.-R., Chung, K.S., Chen, M.-L., Yuan, Z.-M., 2023a. Time-optimal path planning and tracking based on nonlinear model predictive control and its application on automatic berthing. Ocean Eng. 286, 115228. <http://dx.doi.org/10.1016/j.oceaneng.2023.115228>.
- Zhang, T., Zhan, J., Shi, J., Xin, J., Zheng, N., 2023b. Human-like decision-making of autonomous vehicles in dynamic traffic scenarios. IEEE/CAA J. Autom. Sin. 10 (10), 1905–1917. <http://dx.doi.org/10.1109/JAS.2023.123696>.
- Zhao, Y., Han, F., Han, D., Peng, X., Zhao, W., 2022. Decision-making for the autonomous navigation of USVs based on deep reinforcement learning under IALA maritime buoyage system. Ocean Eng. 266, 112557. <http://dx.doi.org/10.1016/j.oceaneng.2022.112557>.